PwC
EU Services

European Commission
Directorate-General for Informatics

Tool Comparison and Recommendations for the SEMIC Style Guide guidelines for RDF and OWL artefacts

SEMIC16 Work Package WP6: Style Guide Updates

Date: May 31, 2026
Doc. Version: 1.0

Document control information

Settings	Value
Document Title:	Tool Comparison and Recommendations for the SEMIC Style Guide guidelines for RDF and OWL artefacts
Project Title:	Semantic Interoperability Supporting Activities 2024-2025
Document Author:	PwC EU Services, Meaningfy
Project Owner:	Georges Lobo
Project Manager:	Catarina Arnaut
Doc. Version:	v1.0
Sensitivity:	Limited
Date:	31-05-2026
Status:	For review by the Contracting Authority

Document Approver(s) and Reviewer(s)

Name	Role	Action	Date
Georges Lobo	EC Business Manager	To Review and Approve	xx-yy-2026
Anastasia Sofou	Reviewer	To Review and Approve	xx-yy-2026
Sander Van Dooren	Reviewer	To Review and Approve	xx-yy-2026

Document history

Revision	Date	Created by	Short Description of Changes
v0.1	23-01-2026	Meaningfy S.à r.l.	Creation of the document and table of contents
v0.2	11-04-2026	Meaningfy S.à r.l.	Incorporation of findings from tool evaluation, drafting recommendations
v0.3	16-04-2026	Meaningfy S.à r.l. and PwC EU Services	Incorporation internal feedback of preliminary draft
v0.4	06-05-2026	Meaningfy S.à r.l.	Incorporation internal review of full draft report
v1.0	31-05-2026	Meaningfy S.à r.l.	Finalisation of report

Configuration Management: Document Location

The latest version of this controlled document is stored in: `https://github.com/SEMICeu/style-guide/`

Executive summary

The SEMIC Style Guide specifies the style for the SEMIC Core Vocabularies and Application Profiles. It is tailored to UML Class Diagrams made with the Enterprise Architect application, which is a proprietary tool with for-payment licence, thereby hindering uptake of CV and AP development. Various community efforts have opted for languages in the World Wide Web Consortium's Semantic Web stack, and they are currently not served well by the Style Guide. The desire is to accommodate this alternative route in the Style Guide.

This raises the question of what the guidelines should be for the RDF/RDFS/OWL/SHACL-focussed approaches. To formulate them, it serves to know what the principles and practices of the current tools are, whether they are alike, to what extent they can be accommodated in prospective new guidelines, and to align with prevalent community practice so as to facilitate uptake, whilst aiming to attain technologically and ontologically sound guidelines.

Therefore, we conducted a thorough comparison to obtain insight into which tool is doing what in order to i) provide the necessary background information and foundations for future reference documentation and justifications, and ii) devise recommendations for follow-up tasks for the Style Guide update, formulate premises towards resolving incompatibilities, and identify several general recommendations for good practices.

The tools and specification selected for comparison were INSPEC, Dataspecer, Yhteentoimivuus, VocBench, model2owl, SEMIC Toolchain, uml2semantics, and the LinkML OWL generator. **The detailed comparison showed that there are a range of different assumptions in the community that are exhibited in the manifold different ways that various features are implemented across the tools and specification**. This includes differences in how knowledge and information is formalised, which language constructs are permitted in which type of artefact, substantive differences in tool design and modelling processes, and attendant issues such as the lack of syntax checking and varied metadata management.

The key differences raised questions and issues, which were formulated into **26 recommendations for follow-up tasks** to address when revising the SEMIC Style Guide, which were accompanied by 5 premises as basis for resolving the issues, and **10 minimal general recommendations that would be useful to discuss, refine, and agree on with the community**. They are included in the deliverable. Recommendations principally concern the architecture and a metamodel for extensible management of the languages for CVs and APs and a better demarcation of what goes in a CV and what in an AP, from which key language decisions can follow and, from there, respective syntax checks. There are also attendant aspects, including metadata and publication guidelines, and recommendations for several editorial improvements.

Contents

List of Figures

List of Tables

1 Introduction

The SEMIC Style Guide [3] specifies guidance on the style that is to be applied to SEMIC's Core Vocabularies (CVs) and Application Profiles (APs). The Guide is tailored to UML class diagrams, and those made with Enterprise Architect (EA) in particular. EA is proprietary and requires a license for payment, which hampers use and broader uptake by the community. In addition, EA's native serialisation changes regularly and its generated XMI file relies heavily on OMG's XMI extension mechanism [46]. The latter is well-known to be practically challenging for interoperability across UML tools [12, 29]. To address this, a UML-to-'something serialised in RDF' tool would need its own modelling language specification that all tool serialisation variants would have to be mapped into. To the best of our knowledge, no such tool exists.

Moreover, there are community efforts with other modelling and serialisation languages to make the models machine-readable, which are not served by the UML guidelines and conventions, notably in the area of the W3C Semantic Web languages including RDF [17], RDFS [15], OWL [42], and SHACL [37]. These languages are standardised with open standards and only RDFS and SHACL have an extension mechanism of sorts, which happens typically only unintentionally for RDFS ending up in the undecidable OWL 2 Full fragment[1]. Thus, overall, this has good prospects of further uptake. Yet, the SEMIC Style Guide contains no guidelines for this route to CV and AP development. This raises the question:

- **What may, could, or should, the Style Guide guidelines be for the RDF/RDFS/OWL/SHACL-focussed approaches to Core Vocabularies and Application profiles?**

In the broader scope of vocabularies and ontologies beyond SEMIC, the question of how to conduct coordinated development of the artefacts is not new and other community efforts in other domains have taken place. This includes mainly the OBO Foundry [48], the SAREF initiative initiated by the European Commission that resulted in a collection of models coordinated by ETSI [24], and recent efforts with the Open Energy Ontology [13]. While these approaches can serve as inspiration, the SEMIC community will have to update and devise guidelines that suits its project and active members, adopters, implementers, and modellers.

[1] RDFS has the feature of being a self-modifying. It mostly happens unintentionally when there's a bug in the code to write OWL syntax or its API's features are not well understood.

To do so, **sub-questions to answer** naturally emerge in order to satisfactorily respond to the main question, being:

1. **What are the principles and practices of the current tools?**
2. **Are they alike and where do they differ?**
3. **To what extent can current practices be incorporated in prospective new guidelines?**

To illustrate potential challenges that need to be cleared up—for the RDF, RDFS, and OWL and the UML-based tools alike—is that there will be formalisation differences [21, 22, 23], some of which can easily be overcome, others not. A simple difference is one tool using `rdfs:label` for the class label and another uses `skos:prefLabel`, which clutters the required additional program code at best. A challenging difference is when one uses RDFS where only domain and range can be specified and another uses the lightweight OWL 2 EL where basic class expressions are specified instead. This is not merely a difference in annotation, but one of semantics. For instance, to link Catalogue to Agent in RDFS, one can only declare the domain and range of the property that relates it:

`domain publisher: Catalogue` and
`range publisher: Agent`.

In OWL, one can omit the domain and range declaration and assert (in shorthand notation):

`Catalogue publisher only Agent`,

which leaves the freedom to use the publisher property also in other contexts, such as books being published by publishers:

`Book publisher only Publisher`.

Such information can be added without affecting the original source, which is typically a CV. In contrast, for an RDFS-only artefact or a modelling style that reuses that modelling practice, the source must be modified too, to:

`domain publisher: (Catalogue or Book)` and
`range publisher: (Agent or Publisher)`,

and that every time publisher is used with new classes, which a user of that CV may not be allowed to do. Moreover, if one CV is designed with the first approach and the other with the second one, the CVs either do not integrate well, have unexpected deductions that are likely to be undesirable, or one cannot be used in tools developed for the other approach, or all of them together. This *lack of interoperability is undesirable and should be prevented from happening*. Common guidelines can help prevent this problem.

The **eventual aim**, therefore, is to devise a set of recommendations for the SEMIC Style Guide for those who use RDF, RDFS or OWL or SHACL without UML as starting point, and such that the recommendations are technologically and ontologically sound and match what as many as the practitioners are doing already, to facilitate their uptake. The **current aims for this report** are:

- to conduct a **thorough comparison to obtain an overview which tool is doing what** exactly,
- to provide the necessary **background information and foundations for future reference documentation and justifications**, and

- to **devise recommendations for follow-up tasks** to realise the Style Guide update that are based on the insights obtained from the comparison, to formulate premises towards resolving any incompatibilities that may emerge, and to **identify general recommendations for good practices** either way.

Therefore, the intended readership of this report is primarily technical experts, except for this introduction and the conclusions (Chapter 5).

The remainder of this report is structured as follows. Chapter 2 describes the materials and methods used to conduct the comparison. The results are summarised in Chapter 3 and extensively discussed in Chapter 4. The recommendations are collated in the main deliverable of the project, and this annexed report closes with final remarks in Chapter 5. One can read this report in that order. The *efficient reading path of least amount of reading*, aside from the executive summary first, is: commence with the recommendations in the deliverable. If they are not evident or the reader would like to read more context: go to Chapter 4. If the analysed results raise questions, examine the results in Chapter 3 and the comparison tables first. If the accompanying text refers to a particular item that the reader thinks they understand but it seems different there, then consult Section 2.1.1 in the Methodology chapter for the description of that comparison feature.

2 Methodology and Materials

This chapter describes the methodology followed for conducting the tool assessment, including the explanations of the features used for the comparison, and brief descriptions of the tools included in the comparison.

2.1 Methodology

The procedure was set out at the start, as follows:

1. Harvest tools and add basic properties (where it is available online, RDF or OWL, or a UML-to-OWL tool, whether it is operational, input and output file formats, and any additional comments), categorise them by type, and prioritise and select tools for comparison;
2. Create a comprehensive and detailed feature comparison list, with a motivation of each feature why is makes sense to include it;
3. Carry out the detailed comparison, including documentation-based and tool-based assessments, and contact the developer where needed (e.g., inquiring about tool access and any recent changes in the light of possibly outdated documentation);
4. Assess the comparison output and discuss commonalities, differences, and questions or problems they raise in the light of the current SEMIC Style Guide;
5. Devise recommendations for future work and for the Style Guide's direction and content.

If the detailed comparison's allocated time appears to be more than needed, a tool may be added and if there's limited time, a tool may be removed from the comparison, or the missing values will be coded with 'unknown' for the remaining features that could not be ascertained within the limited allocated time.

2.1.1 Feature comparison descriptions

This section lists the features that the tools will be compared on, where applicable. The engineering features are listed first and then the details about formalisation options, which is followed by an example of basic effects of differences in the formalisation.

Engineering features

For presentation and readability purposes, the features are grouped into five subgroups: UML serialisation options, disambiguation of RDF, specifics about OWL, modelling patterns, and tool design and features.

UML serialisation options Regarding the UML serialisation options for the UML-to-formal representation tools: EA generates its own serialisation for different EA versions, which amounts to an SQLite relational database system, and it can generate XMI in different versions. Each UML tool is permitted to use the extension part of XMI that may or may not be used depending on the tool and which feature of a tool is used, and there may be tool-specific (non-XMI) serialisations to circumvent the XMI flexibility and typical tool incompatibility.

Disambiguation of RDF RDF can be used in a number of ways, which needs to be disambiguated. Principally, RDF was intended to be used specifically for representing data in a graph. It is meanwhile also used for serialising other models/specifications, notably RDFS (RDF Schema), SHACL shapes, and OWL ontologies. An RDF graph may be valid as a graph, and at the same time be invalid (i.e., violating syntactic restrictions) when it is intended to be a SHACL shapes file or an OWL ontology document. The five interdependent questions for the comparison seek to ascertain how the tool treats the RDF:

- It may be the case (yes/no) that the use of the word RDF means the tool operates in the RDF 'world' such that the files are invalid OWL or they are not guaranteed to be. For instance, it uses the rdflib Python package that does not include in its feature set explicit support for OWL ontologies.
- The 'RDF' is actually RDFS, i.e., it does not contain data (instances) but the contents is a data schema or 'RDF Vocabulary', and thus not operating at the knowledge layer where ontologies reside.
- The 'RDF' is the Turtle serialisation format of OWL and therewith compliant with the OWL standard, and thus the output may be an ontology, an application profile, conceptual data model, or semantic data specification formalised in OWL.
- The previous item can be narrowed down further, on whether the document is within the OWL 2 DL fragment rather than the OWL 2 Full fragment, i.e., that there is no accidental meddling with the language to push it into undecidability.
- The RDF is not merely any arbitrary RDF but adheres to another specific model that is also represented in RDF. For instance, principally or only reusing labels from PROF [9], DSV [36], or SKOS [30].

OWL specifics Given that CVs and APs operate at the type-level and the *de facto* RDF usage (see previous paragraph), it serves to dig deeper into compatibility with OWL, because CVs and APs are not only intended to be used for annotations. First, we determine the OWL species supported using the OWL Classifier[1], which also extracts any declarations that push the produced files into OWL Full or OWL 2 Full, and therewith it can be observed whether they are metadata mistakes that do not affect the meaning of the model itself or whether there are issues with the formalisation (e.g., violating OWL syntax). The DL fragment supported is also determined with the same tool, which provides a better view on what features are actually used compared to the OWL species alone, because one might be in the undecidable OWL 2 Full, yet use very few language features and have a de facto lightweight ontology regarding the content.

The remainder of this set of questions are practically asides and can easily be overcome and would just require extra work for the practitioner. The output format can be any of the 5 recognised serialisation formats of OWL, and likewise for the input format if OWL. Given that not many tools adhere to the required exchange syntax RDF/XML, a conversion option can be convenient for the user and is therefore recoded as well. Other considerations include if and how metadata is converted or maintained from the input file, whether element annotations (e.g., definitions) are converted or permitted, and whether it uses SKOS properties or concepts where OWL itself has relevant features that can be used (e.g., `skos:prefLabel` rather than `rdfs:label`).

Modelling patterns This selection is aimed at the RDF or OWL-based modelling tools, though also can apply to the UML converters. It bears a relation to the formalisation choices (see next section) but conceptually takes a broader view. Among others, whether modelling conventions are described analogous to SEMIC's conventions for UML diagrams, requires a definition for each entity and so on, and, related, a particular naming scheme for the entities, such as class names in capitals. An extension thereto, or in lieu of it, may be the use of predefined ontology design patterns (ODPs) as such or *de facto* in the form of strict templates or forms to fill in to guide content entry. These options may be specified elsewhere and therewith cause built-in dependencies with other files or formats, such as, e.g., PROF [9], DSV [36], SKOS [30], and others.

Lacking such broader views, some may still be gleaned from how content is formalised, of which are selection is included that is also described in the 'Formalisation choices' section below. Specifically, how existential quantification is treated (the common difference is $\exists$ versus ≥ 1), how property inverses are treated (absent, two properties declared as their inverses, or the *inverse()* feature in OWL 2), whether it takes a 'closed view' and generates a domain and range axiom for each object property (association or association end), and whether qualified number restrictions are used.

Tool design and features This part combines several different aspects, some of which are practical, whereas others are consequences of more principled design decisions.

Core tool characteristics include the formalisation approach [23] on whether it uses an algorithmic approach firing one rule after the other for each pattern or if it maps it into its own

[1] `https://github.com/muhummadPatel/OWL_Classifier`

serialisation of the UML class diagram or similar semantic specification and then converts, and either can be computed on the fly during the creation or is updated only in batch mode. The batch mode is more amenable to include a step where the formalisation options may be configurable, be it by type of artefact that is being developed or some other mechanism, such as a toggle function to declare classes automatically disjoint, or that one may be able to choose the OWL species for the formalisation. Last, a core design option is whether it uses an internal metamodel, which makes a tool more easily maintainable and extensible at least by design.

Other features include checking whether metadata, such as author or version, can be declared, and if so, whether it uses the Semantic Registry Model (SRM) [10] of SEMIC or not and whether it has any constraints or 'conventions' on UML diagrams that are being checked in the process, be they SEMIC conventions or other written restrictions or additions. Other practical aspects are whether, and if so, how, it imports other vocabularies or ontologies, which can be either:

- Hard import, be this of the complete artefact referenced or a defined module thereof. In both cases, this occurs with an `owl:import` statement, but the URI of the import file differs. In case of the complete artefact, then each time the ontology is loaded, the latest version of the imported artefact is dynamically consulted. In case of a module, it fetches the information from a different location where that module is stored and available, and may thus eventually diverge from the origin if the origin is updated.
- Soft import, which involves 'copying in' the vocabulary and relevant axioms from the other artefact and, if applicable, the namespace is added to the list of prefixes. This severs any connection to the source artefact and may thus eventually diverge from the origin if the origin is updated. An advantage is that any alignments made do not break as soon is the source changes in content or IRI without persistent URI management.

Related to the imports are what input formats the tool accepts, not only for the UML files (e.g., different XMI versions, XMI from different tools), but also for the RDF and OWL files. For instance, if the tool requires RDF/XML format but the vocabulary is published only in Turtle, or vice versa, then there are additional manual preprocessing steps needed for use. Such use and reuse consideration also motivate recording the output file format, especially if it has been shown to be in OWL but the file has only a `.ttl` or `.rdf` file extension (indicating RDF only) instead of `.owl` (indicating that the RDF complies with OWL syntax).

Additionally, we check whether the tool has a built-in reasoner, be it serving as a quality check to determine that the UML-to-OWL conversion generated a coherent ontology, or for other uses, such taxonomic classification or use in ontology-based data access. This has as prerequisite checking whether the tool generates syntactically correct OWL and whether the generated file can be processed by other tools. For this, we use both the permissive Protégé v5.6.5, which tries to 'second-guess' corrections when it encounters issues parsing the input file and provides a detailed report in the log file, and the strict OWL Classifier.

Some tools may deem OWL out of scope, and therefore it is also recorded whether the tool converts UML or UML-like constraints to SHACL shapes, be it in parallel to OWL or instead of it. An obvious additional feature check is the API usage for these file formats, also because it indicates whether OWL may be supported and how, and the reliability of the

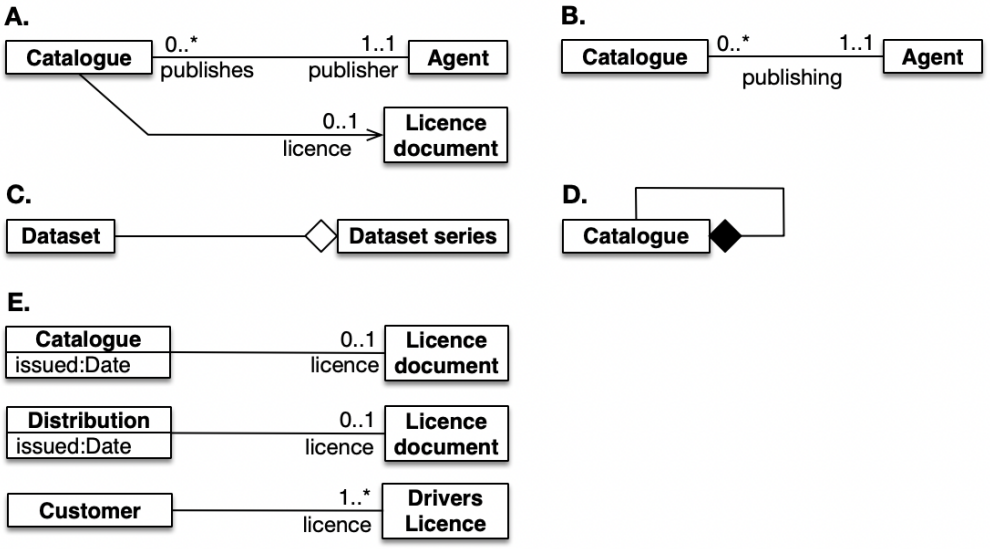

Figure 2.1: Sample diagrams for illustrations; see Section 2.1.1 for the cases.

conversions. Notably, the OWP API has a long history [28] and is widely assumed to be free of errors by now, Owlready [39] works with Python but does not support all OWL 2 DL language features, and rdflib[2] does not support OWL although it can generate documents that use at least some of OWL's reserved vocabulary.

We also added whether the tool offers visual modelling, i.e., editing the model and not just viewing, offers only a view of the model, be this static or dynamic diagrams with moving parts, or is text-based only.

Formalisation choices from UML to OWL and/or generally

The list of UML features was reused from [34] and the possible values are assumed self-explanatory for those who know the UML standard [45] regarding UML class diagrams. The remainder of this section focuses on the formalisation choices and the myriad of alternative options that may or may not be logically equivalent. The following abbreviations are used: Val.: possible values for that feature; Expl.: explanation of the feature and why it matters; and Ex.: example illustrating the different options. The illustrative examples are mostly from DCAT-AP, where possible.

A number of these features also can be understood as decisions independent from UML; e.g., item 1 is relevant also for structured controlled vocabularies such as the Gene Ontology that uses ID+labels and lightweight ontologies such as SNOMED CT that uses element names, which are not outputs of UML-to-OWL conversions but direct modelling decisions in the target file.

1. Element naming/vocabulary.

   Val.: element names, labels + IDs

[2] `https://rdflib.readthedocs.io/en/stable/`

Expl: UML is name-based, as is OWL by default. The ID+labels approach spilled over from GO/OBO practices of OBOEdit.

Ex.: A UML class *Person* is formalised into either:

a) An OWL class Person, or

b) An OWL class CBV:00012345 + label Person that by default uses `rdfs:label`.

2. One or both association ends are bumped up to object property.

   Val.: no, yes (one, both)

   Expl.: It is one of the variants for bridging the positionalist to the standard view of what relationships are ontologically (see [23] section 3.1 for an explanation).

   Ex.: If yes: two UML classes *Catalogue* and *Agent* with association end *publisher* on the *Agent* side and *publishes* on the *Catalogue* side (see Figure 2.1-A) is formalised into:

   a) two object properties: publisher and publishes, or

   b) either publisher or publishes is added to the logical theory, arbitrarily or based on declared multiplicity (typically the end with an existential quantification/mandator multiplicity).

3. Both association ends are bumped up to object properties and declared inverses.

   Val.: no, yes

   Expl. This is conditional on a 'yes, both' answer to item 2 and relevant in the light of OWL 2 EL expressivity because it has no inverses.

   Ex.: If yes: as in item 2a together with the axiom $\text{publisher} \equiv \text{publishes}^{-}$.

4. Association to object property.

   Val.: yes, no

   Expl.: It is another variant for the bridging positionalist to standard view of what relationships are ontologically (see [23] section 3.1 for an explanation), and depends on the modelling practices used in the UML class diagram or its look-alike.

   Ex.: Two UML classes *Catalogue* and *Agent* with association *publishing* (see Figure 2.1-B) is formalised into an object property, either

   a) With the same name, publishing, or

   b) NLP is applied and publisher and/or publishes is generated from the gerund.

5. Shared aggregation (approximation).

   Val.: plain object property (with part-of/has-part name), object property with property characteristics (namely, ...)

   Expl.: For shared aggregation/part-of, the property characteristics would be transitivity and reflexivity; if any one is declared, transitivity is the most common of the two. Note 1: parthood is not a primitive in the DL-based species of OWL, so it is always approximated in some way by means of an object property. Note 2: declaring the property transitive makes it 'non-simple' and it then it cannot be used in a qualified number restriction (among others).

Ex.: Consider *Dataset* as a member of a *Dataset Series* (see Figure 2.1-C), then there are several options:

a) Introduce new vocabulary partOf or a notational variant thereof, assert that Dataset $\sqsubseteq \forall$partOf.DatasetSeries;
b) Introduce new vocabulary hasPart or a notational variant thereof, assert that DatasetSeries $\sqsubseteq \forall$hasPart.Dataset;
c) as in a), above, plus Trans(partOf);
d) as in b), above, plus Trans(hasPart);
e) as in item 3 (i.e., add partOf and hasPart, declare partOf $\equiv$ hasPart$^-$) and add either DatasetSeries $\sqsubseteq \forall$hasPart.Dataset or Dataset $\sqsubseteq \forall$partOf.DatasetSeries; or
f) as in e), and add either c) or d) as well.

6. Composite aggregation (approximation).

Val.: plain object property (with proper-part-of/has-proper-part name), object property with property characteristics (namely, ...)

Expl.: For composite aggregation/proper-part-of, the property characteristics should be transitivity, irreflexivity, and asymmetry, but they cannot be declared all three at the same time [35]; if any one is declared, transitivity seems to be the most common of the three. Note: parthood is not a primitive in the DL-based species of OWL, so it is always approximated in some way by means of an object property, and declaring it transitive makes in a 'non-simple' property.

Ex.: The options are similar to item 5, but then for proper parthood. Consider *Catalogue* having another catalogue as proper part (see Figure 2.1-D), then there are the following options:

a) Introduce new vocabulary properPartOf or a notational variant thereof, assert that Catalogue $\sqsubseteq \forall$properPartOf.Catalogue;
b) Introduce new vocabulary hasProperPart or a notational variant thereof, assert that Catalogue $\sqsubseteq \forall$hasProperPart.Catalogue;
c) as in a), above, plus Trans(properPartOf);
d) as in b), above, plus Trans(hasProperPart);
e) as in a), above, plus Asym(properPartOf) or Irr(properPartOf);
f) as in b), above, plus Asym(hasProperPart) or Irr(hasProperPart);
g) as in item 3 (i.e., add properPartOf and hasProperPart, declare properPartOf $\equiv$ hasProperPart$^-$) and add either Catalogue $\sqsubseteq \forall$hasProperPart.Catalogue or Catalogue $\sqsubseteq \forall$properPartOf.Catalogue; or
h) as in e), and add either c) or d) as well.

7. Navigability of the association (line with arrow on one side).

Val.: 'approximated' with declaring an object property in one direction, ignored, error

Expl.: Navigability is not supported in OWL because it concerns object behaviour. Navigability is intended in UML for security/behaviour to govern which object can access what, but practically in the Style Guide conventions [4] it is used to indicate whether the association has only one named association end and to pretend

assigning directionality to an association that is non-directional by design (see [20] for a practical elaboration on the technical and philosophical distinction).

Ex.: If it is approximated, then given, e.g., two UML classes *Catalogue* and *Licence Document* with association end *licence* and coincidentally that multiplicity constraint (see Figure 2.1-A), one may expect it to be converted into Catalogue $\sqsubseteq \leq 1$ licence.LicenceDocument without a reported error that the other end is missing a name.

8. Dependency association (dashed line with arrow on one side).

   Val.: 'approximated' with declaring an object property in one direction, ignored, error

   Expl.: Dependency is not supported in OWL. Dependency in UML alludes to the idea of weak entity types in EER, and the dependant entity (on the arrow side) depends on the main entity. As per SEMIC Style Guide [4], it is abused for dealing with enumerations where, in fact, the 'dependant entity' is an independent vocabulary list.

   Ex.: If it is approximated, then it should be treated as a plain association and formalised like navigability's approximation as described in item 7.

9. Domain and range axiom for each association or association end that is formalised into an object property.

   Val.: yes, no

   Expl.: This option is also called typing of the relationship and is likely too constrained for a Core Vocabulary, and certainly so for an ontology. It might be useful for an AP that is not intended to be reused in other APs.

   Ex.: The association or association end is converted as in item 2, 3, or 4 (see Figure 2.1-A), and then add both:
   - $\exists$publisher.$\top \sqsubseteq$ Catalogue ('the domain of publisher is catalogue' or 'if there's an outgoing publisher relation then it originates from catalogue'), and
   - $\top \sqsubseteq \forall$publisher.Agent ('the range of publisher is agent' or 'all objects that have an incoming relation publisher are agent').

10. Association or association end to object property with a class expression axiom instead of domain and range axioms.

    Val.: yes, no

    Expl.: Should be yes, given the assumption that associations are not unique in the model and will not result in differently name object properties in the logical theory.

    Ex.: Association or association end converted as in item 2, 3, or 4 (see Figure 2.1-A), and add: Catalogue $\sqsubseteq = 1$ publisher.Agent (with the exact cardinality accidental to the example).

11. Multiplicity to qualified cardinality.

    Val.: yes, no

Expl.: Should be yes, given the assumption that associations are not unique in the model and will not result in differently name object properties in the logical theory, and that there are no expressivity constraints so that OWL 2 DL is permitted.

Ex.: Consider again the Catalogue, publisher, and Agent:

a) A 'yes' results in Catalogue $\sqsubseteq = 1$ publisher.Agent (as in item 10).

b) A 'no' will result in a formalisation alike item 12.

12. Multiplicity to unqualified cardinality + domain and range axioms.

Val.: yes, no

Expl.: Should be with a class expression instead, as in item 11, under the assumption that associations are not made unique in the logical theory, and/or under the assumption that it is not merely a formalisation of the UML diagram for one application but also intended for reuse.

Ex.: Consider again the Catalogue, publisher, and Agent:

a) A 'yes' means using domain and range axioms as in item 9 and adding Catalogue $\sqsubseteq = 1$ publisher.$\top$.

b) A 'no' here likely will be the same as a 'yes' in item 11.

13. N-aries where n>2 approximated or ignored.

Val.: approximated, ignored, error

Expl.: Ideally, they are approximated when n-aries are not available.

Ex.: There are a number of ways to approximate n-aries, the essential components being creating a class out of the n-ary and n new binary object properties that relate the new class to the n participating classes, and each with an 'exactly 1' class expression axiom. Consider the n-ary in Figure 2.2-A with *Customer*, *Bike*, and *City*, then there are several options, neither equally good nor logically equivalent:

a) Basic: i. create a new class either manually or by some automated way (let's assume BikeRental), and simply convert the other classes into Customer, Bike, and City; ii. use a method to convert the association ends, where here we assume association end to object property and for any missing association and name, we use the hasX pattern, so we obtain three object properties: rents, in, and hasCustomer; iii. declare the mandatory 1:1 for each: BikeRental $\sqsubseteq = 1$ hasCustomer.Customer, BikeRental $\sqsubseteq = 1$ in.City, and BikeRental $\sqsubseteq = 1$ rents.Bike.

b) Like a), above, but rather introduce three new object properties solely used for the reification, r1, r2, and r3, which are not allowed to be used elsewhere in the logical theory.

c) Like in a) or b), above, but carry it out an instantiation of an ontology design pattern (ODP), which typically also adds annotations for traceability.

14. Attribute to data property.

Val.: with class/domain, with datatype/range (specific datatype, literal/anytype), domain & range both declared, only data property

Expl.: More or less of UML's attribute can be ported to OWL, with a ready source of potential conflict the datatype mismatches. The UML standard permits a few built-in datatypes and extensions [45], whereas OWL requires datatypes to be one of a specific subset of XML data types [42] or `rdfs:Literal`. The differences can be bridged by approximate mappings in a conversion script or by converting all data types to `rdfs:Literal` or to leave the data property range empty. Then there is the issue of attribute uniqueness versus DP reuse (elaborated on in item 15).

Ex.: Consider the attribute *issued* of class *Catalogue* in Figure 2.1-E, with datatype `Date`. OWL supports `xsd:dateTime` and `xsd:dateTimeStamp`, so UML's data type will need to be mapped to either one of them or to the generic `rdfs:Literal` or to `rdf:XMLLiteral`. Then there are the following main variants:

a) Just the attribute: *issued* is formalised into the issued data property in OWL;
b) Declare the domain: $\exists$issued.$\top \sqsubseteq$ Catalogue;
c) Declare the range to a specific datatype $\top \sqsubseteq \forall$issued.`xsd:DateTime`;
d) Declare the range to a generic datatype $\top \sqsubseteq \forall$issued.`rdfs:literal`;
e) Both b)+c) or b)+d), above;
f) Convert attributes into object properties or classes, such as using DOLCE's Quality and qualia (see [32] chapter 6 for an explanation), which is not elaborated on here given that this is currently a manual process (the converse is automated [18]).

15. Object property and Data property reuse.

Val.: yes (OP only, DP only, both), no

Expl.: In OWL, a vocabulary element can be used throughout the logical theory, both in the case of object properties and data properties. While this is the case for UML associations as well (albeit not convincingly clear), UML's attributes have a local scope within the UML class. Harmonising the underlying assumptions can be achieved in different ways. From a use and reuse perspective, OP and DP reuse—and, hence, minimising the number of vocabulary elements—is preferred.

Ex.: Consider the UML diagram snippet in Figure 2.1-E: *licence*—be it as an association end that will be bumped up to an OP or as an association to OP—appears twice relating different classes, and the attribute *issued* is used inside each of *Catalogue* and *Distribution*. There are four principal options:

a) If they are *not* reused, then new OPs/DPs have to be created with different names for different classes. The common approach is to append the class' name. For this example, because *licence* points twice to the same class but we need to distinguish them, we obtain three object properties with awkward names, such as: hasLicenceLicenceDocument, hasLicence1LicenceDocument, and hasLicenceDriversLicence.
b) If they *are* reused and domain & range axioms were declared as per item 9, then a union of classes have to be declared in the domain or range. For this ex-

ample, we obtain licence with as domain $\exists$licence.$\top \sqsubseteq$ Catalogue $\sqcup$ Distribution $\sqcup$ Customer and range $\top \sqsubseteq \forall$licence.(LicenceDocument $\sqcup$ DriversLicence).

c) If they *are* reused and associations/association ends use the transformation approach as in item 2, 3, or 4, then they also use the class expression axioms as in item 10. For this example, we obtain licence and three class expressions: Catalogue $\sqsubseteq \leq 1$ licence.LicenceDocument, Distribution $\sqsubseteq \leq 1$ licence.LicenceDocument, and Customer $\sqsubseteq \exists$licence.DriversLicence.

d) If they *are* reused but the modeller does not want that, then a workaround is used through naming conventions in the UML class diagram upfront, which is propagated into the formalisation, typically with hasX/isOfX in association ends (with X the class's name) or a synonym is used.

16. Mandatory multiplicity (1..* and the first 1 in a 1..1 in the UML class diagram) and existential quantification.

    Val.: min 1, $\exists$

    Expl.: It should be $\exists$. While they are conceptually equivalent, the former requires the qualified cardinality constraint language feature whereas the latter does not. For OWL, the 'min 1' pushes the required language from OWL DL into OWL 2 DL, and has other unintended consequences, such as conflicts with transitivity of properties and property chains (disallowed together with qualified cardinality constraints), and pushes reasoning complexity in N2ExpTime.

    Ex.: Consider Figure 2.1-E with the customer's drivers licence and assuming a transformation with class expression (item 10) for ease of illustration, then there are the following two options:

    a) Customer $\sqsubseteq \geq 1$ licence.DriversLicence; and
    b) Customer $\sqsubseteq \exists$licence.DriversLicence.

17. Multiplicity of exactly 1 in the formalisation.

    Val.: exactly 1, $\exists$ + functional

    Expl.: It should be exact cardinality, which captures the intent more precisely and in a way such that the properties more easily can be reused without obtaining undesirable deductions. The $\exists$+functional is a workaround approximation when qualified cardinality is not available in the language. Declaring the property functional imposes the constraint on all its uses even though it may not hold everywhere in the logical theory. That second 1 in the 1..1 in a UML class diagram is locally to that association end, not to all its occurrences in the model.

    Ex.: Consider Figure 2.1-A: while a catalogue may have at most one agent for publisher, a book author may well have more than one agent as publisher, which would be conflicting with the functional assertion of publisher. The three options are as follows (incorporating also the two variants of item 16 that may interfere):

    a) Catalogue $\sqsubseteq = 1$ publisher.Agent;
    b) Catalogue $\sqsubseteq \exists$publisher.Agent and Func(publisher); or, worse:
    c) Catalogue $\sqsubseteq \geq 1$ publisher.Agent and Func(publisher).

18. Optional/multiplicity of 0..* or 0..1 in UML diagram and universal quantification.

    Val.: domain/range axiom, min 0, $\forall$, functional

    Expl.: Preferably a class expression with universal quantification ($\forall$) for wider reuse. The 'min 0' has the same computational issue with qualified cardinality as the 'min 1' (see item 16). Declaring the property functional has the same issue as described in item 17 for the maximum cardinality in the 'the exactly 1'. The domain/range option was described as part of item 9, and that it is more constraining than asserted in UML (see item 15 on object property reuse and item 10 on class expressions).

    Ex.: Consider Figure 2.1-A for an *Agent* that *publishes* 0..* *Catalogue*s and Figure 2.1-E where *Catalogue* has *licence* 0..1 *Licence Document*s, then:

    a) The 'min 0' option: Agent $\sqsubseteq \geq 0$ publishes.Catalogue;
    b) The $\forall$ option: Agent $\sqsubseteq \forall$publishes.Catalogue;
    c) The domain and range option: see axioms in item 9;
    d) The functional option if it is 0..1, with qualified cardinality: Catalogue $\sqsubseteq \leq 1$ licence.LicenceDocument;
    e) The functional option if it is 0..1, with functional assertion: domain & range analogous to item 9 and Func(licence).

19. Classes outside the hierarchy are declared disjoint.

    Val.: yes (disjoint, complement), no

    Expl.: It should be yes. By default, classes in a UML class diagram are mutually disjoint, unless they appear in a class hierarchy, where they have to be declared disjoint explicitly if they are. This assumption may or may not have been carried over into the logical theory. Second, there is a subtle distinction between complement ($C \sqsubseteq \neg D$ 'each C is not a D') and disjointness ($C \sqcap D \sqsubseteq \bot$ 'the intersection of C and D is empty').

    Ex.: Consider the model snippet in Figure 2.2-C with *Distribution*, *Resource*, and its two subclasses *Dataset* and *DataService*, which typically has either one of the following formalisation practices:

    a) yes, complement: Distribution $\sqsubseteq \neg$Resource and Dataset $\sqsubseteq$ Resource and DataService $\sqsubseteq$ Resource are declared in the theory;
    b) yes, disjointness: Distribution $\sqcap$ Resource $\sqsubseteq \bot$ and Dataset $\sqsubseteq$ Resource and DataService $\sqsubseteq$ Resource are declared in the theory;
    c) no: only Dataset $\sqsubseteq$ Resource and DataService $\sqsubseteq$ Resource are declared in the theory.

20. Stereotypes for types.

    Val.: As a new superclass/OP, other (namely, ...), not supported (ignored, error)

    Expl.: Under the assumption that stereotypes are used in the UML class diagram to indicate categories, and possibly those from a foundational ontology, then they would conceivable end up as their supertype in the logical theory. Other options

are possible as well, such as extra-logical processing or ontological reasoning services, such as with OntoUML [26]. If they appear in the model, then ideally either of the first two options would be the case where at least the first option affects the outputted axioms.

Ex.: Consider the model snippet in Figure 2.2-D with the stereotype ≪*Role*≫ for the class *Agent*, which could possibly be from the DOLCE or BFO ontology or another one that then has to be imported into the ontology. Some of the options, briefly:

a) If the stereotype is added as a class in the ontology as a supertype of the class that is stereotyped, then this stereotype use forces creation of a new class Role and addition of Agent $\sqsubseteq$ Role.
b) Other options include: i) add the stereotype as an annotation to the class Agent, or ii) resort to second order logic, such as with DOL [5].
c) If it is ignored, then only Agent is added.

21. Stereotyes for modalities.

Val.: description added in annotation, squeezed in the OP/DP name, ignored, error, formalised elsewhere

Expl.: Stereotypes for ≪*mandatory*≫ and ≪*optional*≫ are redundant given the respective multiplicity constraint, and thus should have no effect anywhere, other than possibly redundantly adding it as an annotation for later processing (e.g., automated HTML documentation generation), or adding it to a property's name (highly discouraged). Stereotypes such as ≪*recommended*≫, ≪*should*≫ ≪*at some time*≫ etc. are beyond OWL expressivity and also should not be appended to a property's name because it has no logical consequences, yet may give an end user modeller a false sense of adequate representation. It can be formalised outside OWL whilst not abandoning OWL altogether, principally by availing of DOL [5], where it can be added in a first-order predicate logic module.

Ex.: An example of a stereotype to indicate modality on a constraint is shown in Figure 2.2-D, where *publisher* has multiplicity 0..1, i.e., it is optional, but annotated with ≪*recommended*≫.

a) Push it in an annotation: add publisher, add a suitable 'at most one' formalisation (see item 18), and add "recommended" in a `rdfs:comment` field.
b) Formalise it elsewhere: if using DOL, then the whole theory without the modality is imported into a new one that uses a language expressive enough to represent the modality and where the suitable modality is added as an additional axiom.
c) Modify the property name: *publisher* is added to the ontology in either of the aforementioned ways of formalising associations and attributes, but the name changed into recommendedPublisher.

22. Uses a foundational ontology.

Val.: no, possible, yes (namely, ...)

Expl.: A foundational ontology may be useful in a number of ways, including aligning the CVs to one to foster interoperability across CVs and to guide the modelling processes, to enforce constraints at the back-end as an ontology-driven system, or to aid in developing an internal metamodel for the tool to manage multiple formats.

Ex.: An example has not been added due to time constraints and the reader is encouraged to explore extant proposed uses, which range from adding ontological notions to a language (e.g., OntoUML [26]) to foundational ontology-guided domain modelling guidance (e.g., OntoPartS [33] and the BFO Classifier [11]).

23. Advanced language features.

Val.: OP property characteristics (if yes, which), property chain, GCI, one-of, 'key'

Expl.: They are not used pervasively in OWL files in general and nominals/one-of and 'key' may have been misused. OWL's one-of could be used for an enumeration and `hasKey` could be misused for an *{id}* in UML. OWL's 'has key' operates on individuals, however, and is only at most 1, not mandatory 1:1.

Ex.: Consider the *Monetary Value* with *c-voc:currency* example in Figure 2.2-E, where the ≪enumeration≫ stereotype is a special data type denoted in extended notation, stating that each monetary value is in exactly one currency taken from a controlled list of currencies abbreviated here as *c-voc:currency* (e.g., EUR, CHF, CZK, DKK, GBP etc.), although they could have been listed there instead.

a) Nominals with one-of (`ObjectOneOf`): this involves: i. converting the datatype into a class (i.e., Currency); ii. converting the entries of the controlled list into individuals in OWL and declaring them disjoint[3] and instances of the newly created class (Currency(EUR) etc.); and iii. declaring the class equivalent to the set of individuals (Currency $\equiv$ {EUR} $\sqcup$ {CHF} $\sqcup$ {CZK} $\sqcup$ {DKK} $\sqcup$ {GBP}).

b) Nominals, partial conversion: as in a), but then without the disjoint individuals. The issue with this is that it does not fully capture the intent, because OWL operates under no UNA (unique name assumption) and so it might infer that, e.g., `ex:EUR` and `ex:CHF` are the same individual. The UNA is simulated by explicitly adding disjointness axioms among the individuals.

c) Classes and instances only: convert the datatype into a class (Currency), convert the entries of the controlled list into individuals in OWL, declare them disjoint and instances of that new class.

d) Classes and instances, foundational ontology-inspired: convert the datatype into a class (Currency) which is made a subclass of, e.g., dolce:quality, create a new class, e.g., CurrencyType as a subclass of one of DOLCE's dolce:Abstract classes (e.g., abstract region in case of currencies) and convert the entries of the controlled list into classes in OWL.[4]

e) Partially outside OWL: if the artefact is intended to be part of an ontology-driven information system and not also be useful elsewhere, one may opt for

[3] With abuse of notation: EUR $\neq$ CHF, EUR $\neq$ CZK etc.

[4] For currencies this could make sense in that some are fiat currencies, others are backed by gold, others are cryptocurrencies, abstract 'stones' in LETS etc.

storing the controlled list internally in an array, in a database table, a csv file or the like, rather than importing the list in the ontology, and convert only the name of the vocabulary, either as an assertion with a value or as a class.

24. Abuse of notation in conventions for UML class diagrams.

    Val.: no, yes (namely, ...)

    Expl.: This may be to overcome tool limitations or implementation difficulties or to use language features that are not natively in UML according to the standard.

    Ex.: Notable options include:
    a) Abusing navigability to pretend there is a directionality to the association, which aligns with the directionality requirements of OWL; and
    b) Abusing the dependency association to related classes to external controlled lists.

25. Requires adherence to UML class diagram modelling conventions for the transformation to work.

    Val.: yes, no

    Expl.: Designing transformation algorithms for arbitrary serialised UML diagrams has many corner cases that complicates the rules and algorithms. Conventions are a way to make this process more manageable from the view of software development.

    Ex.: The SEMIC Style Guide's conventions for UML class diagrams [4] or the OSLO Modelleringsregels [6].

26. Tweaks to UML constructs in modelling conventions.

    Val.: yes (restricts UML, changes semantics), no

    Expl.: This is conditional on 'yes' to the previous question and then intersects with item 24. Support for a fragment of UML is not uncommon and not necessarily problematic, especially when it concerns features that are not supported in the target logic anyway. If the tweaks involved changes in the semantics, then it affects compatibility of the tooling negatively, locking it into the conventions the tool adheres to.

    Ex.: The examples in item 24 are changes in the semantics compared to how they are defined in the UML specification [45], yet are included in the SEMIC Style Guide's conventions for UML class diagrams.

Example: formalisation differences

We illustrate differences in formalisation choices for various language features, and consequent different serialisation even when they use the same syntax (be that for OWL or the SHACL shapes that would be generated). The small UML class diagram shown in Figure 2.3 uses parthood and subsumption, the same association and association ends twice, and

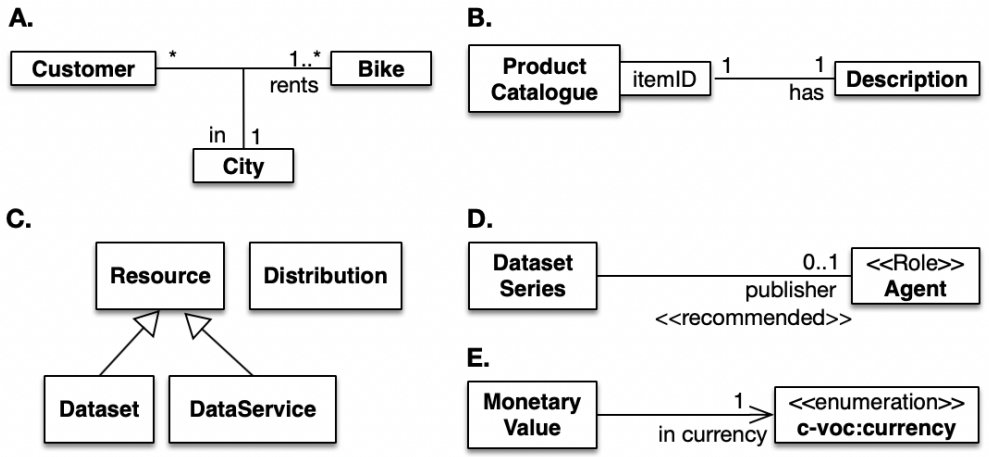

Figure 2.2: More sample diagrams for illustrations; see Section 2.1.1 for the cases.

three distinct multiplicities. Two 'styles' are used in transforming the sample model into Turtle syntax: one of ontology with an 'open view' of use and reuse across applications (left) and one of a 'closed view' for a model in one context of a single application (right) where the UML diagram notation may be used as an information model or conceptual data model rather than as a visualisation of an ontology or vocabulary for interoperability. This is not the same as OWA and CWA, which affects inferencing, but rather the intent, which affects modelling style and content.

The ontology/reuse style uses class expressions rather than domain and range axioms, implied property inheritance, and the common assumption that parthood is transitive, which it is in mereology. The conceptual data modelling style assumes there is nothing relevant beyond what is explicitly modelled in the UML class diagram, and so it limits or 'closes' each property's domain/range to only the classes and attributes declared in the model. The respective formalisations are *not* logically equivalent and can easily result in different deductions as-is and when more knowledge is added.

Besides the DL notation of the axioms (see Figure 2.3), we include the Turtle notation of each option, with the ontology/vocabulary style in Listing 2.1 and the conceptual data modelling style in Listing 2.2. For instance, compare the domain and range axioms (with a union of classes) in Listing 2.2 lines 8-14, 18-21, and 27-28 and a 'min cardinality 1' in line 38, with Listing 2.1 that has only the classes expression axioms (lines 20-44) and `someValuesFrom` instead of 'min 1' (line 24).

The respective DL fragments are similar, as shown in Figure 2.4 and Figure 2.5, respectively.

Listing 2.1: Ontology or reuse style of formalising the UML class diagram, relevant sections (omitting the metadata and prefix declarations)

```
1  ###################################################################
2  #    Object Properties
3  ###################################################################
4
5  ###  http://www.umlintoowlex.org/semicex#aeA
6  :aeA rdf:type owl:ObjectProperty ;
```

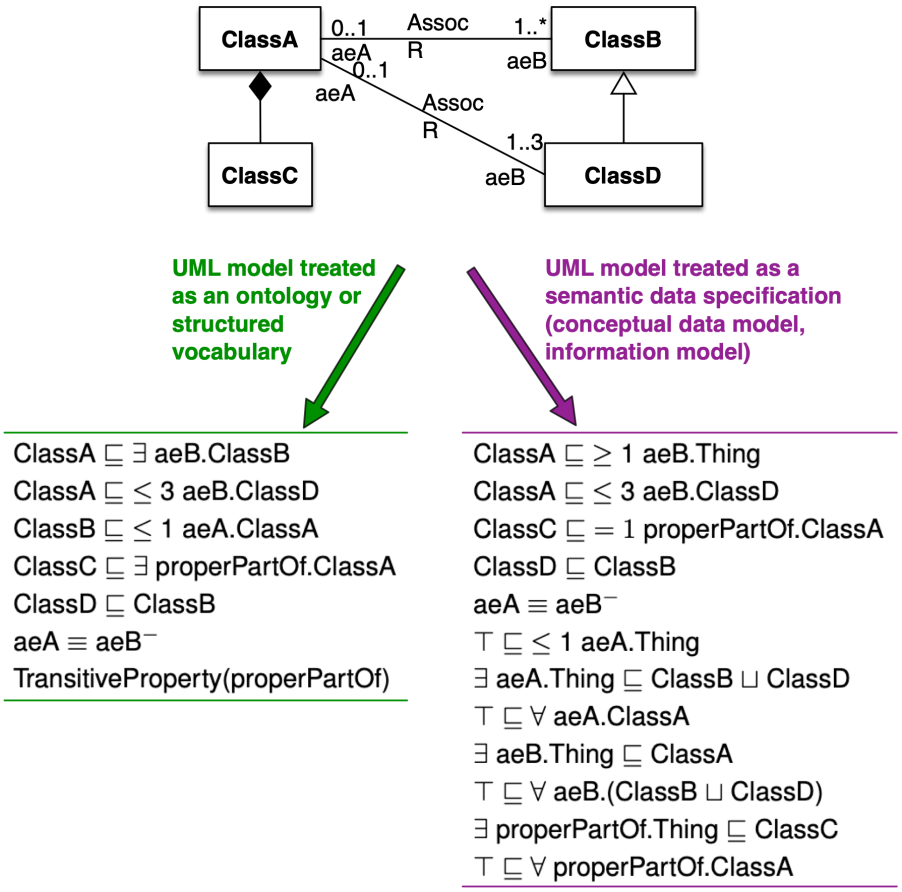

Figure 2.3: Example UML class diagram to illustrate model conversion differences. Left: in the spirit of ontologies and reuse; right: in the spirit of conceptual data model or application profile form one application. See Section 2.1.1 for details.

```
7       owl:inverseOf :aeB .
8
9   ###  http://www.umlintoowlex.org/semicex#aeB
10  :aeB rdf:type owl:ObjectProperty .
11
12  ###  http://www.umlintoowlex.org/semicex#properPartOf
13  :properPartOf rdf:type owl:ObjectProperty ,
14                  owl:TransitiveProperty .
15
16  #################################################################
17  #    Classes
18  #################################################################
19
20  ###  http://www.umlintoowlex.org/semicex#ClassA
21  :ClassA rdf:type owl:Class ;
22          rdfs:subClassOf [ rdf:type owl:Restriction ;
23                  owl:onProperty :aeB ;
24                  owl:someValuesFrom :ClassB
25                ] ,
26                [ rdf:type owl:Restriction ;
27                  owl:onProperty :aeB ;
```

```
28                  owl:maxQualifiedCardinality "3"^^xsd:nonNegativeInteger ;
29                  owl:onClass :ClassD
30               ] .
31
32  ###  http://www.umlintoowlex.org/semicex#ClassB
33  :ClassB rdf:type owl:Class ;
34          rdfs:subClassOf [ rdf:type owl:Restriction ;
35                  owl:onProperty :aeA ;
36                  owl:maxQualifiedCardinality "1"^^xsd:nonNegativeInteger ;
37                  owl:onClass :ClassA
38                ] .
39
40  ###  http://www.umlintoowlex.org/semicex#ClassC
41  :ClassC rdf:type owl:Class ;
42          rdfs:subClassOf [ rdf:type owl:Restriction ;
43                  owl:onProperty :properPartOf ;
44                  owl:someValuesFrom :ClassA
45                ] .
46
47  ###  http://www.umlintoowlex.org/semicex#ClassD
48  :ClassD rdf:type owl:Class ;
49          rdfs:subClassOf :ClassB .
```

Listing 2.2: Information/Conceptual data modelling style, of formalising the UML class diagram, relevant sections (omitting the metadata and prefix declarations)

```
1   ###################################################################
2   #    Object Properties
3   ###################################################################
4
5   ###  http://www.umlintoowlex.org/semicex#aeA
6   :aeA rdf:type owl:ObjectProperty ;
7        owl:inverseOf :aeB ;
8        rdf:type owl:FunctionalProperty ;
9        rdfs:domain [ rdf:type owl:Class ;
10                owl:unionOf ( :ClassB
11                              :ClassD
12                            )
13             ] ;
14       rdfs:range :ClassA .
15
16  ###  http://www.umlintoowlex.org/semicex#aeB
17  :aeB rdf:type owl:ObjectProperty ;
18       rdfs:domain :ClassA ;
19       rdfs:range [ rdf:type owl:Class ;
20                owl:unionOf ( :ClassB
21                              :ClassD
22                            )
23             ] .
24
25  ###  http://www.umlintoowlex.org/semicex#properPartOf
26  :properPartOf rdf:type owl:ObjectProperty ;
27                rdfs:domain :ClassC ;
28                rdfs:range :ClassA .
29
```

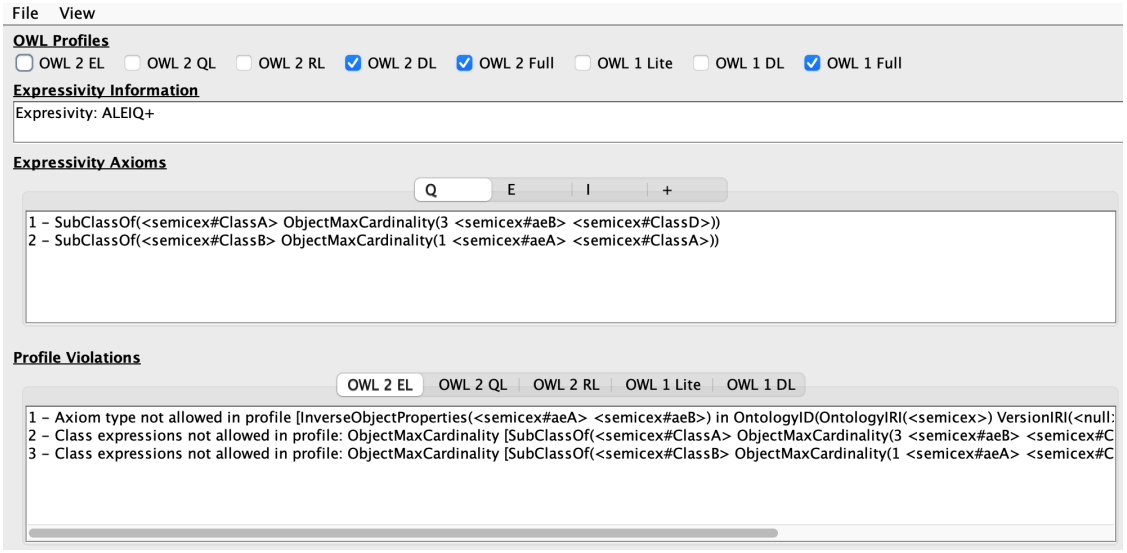

Figure 2.4: DL fragment and OWL species membership for the 'ontology/reuse' style of UML class diagram model conversion of the diagram in Figure 2.3 and in Listing 2.1 (RDF/XML serialisation used), as in the OWL Classifier v1.3.

```
30 ###################################################################
31 #    Classes
32 ###################################################################
33
34 ###  http://www.umlintoowlex.org/semicex#ClassA
35 :ClassA rdf:type owl:Class ;
36         rdfs:subClassOf [ rdf:type owl:Restriction ;
37                 owl:onProperty :aeB ;
38                 owl:minCardinality "1"^^xsd:nonNegativeInteger
39               ] ,
40               [ rdf:type owl:Restriction ;
41                 owl:onProperty :aeB ;
42                 owl:maxQualifiedCardinality "3"^^xsd:nonNegativeInteger ;
43                 owl:onClass :ClassD
44               ] .
45
46 ###  http://www.umlintoowlex.org/semicex#ClassB
47 :ClassB rdf:type owl:Class .
48
49 ###  http://www.umlintoowlex.org/semicex#ClassC
50 :ClassC rdf:type owl:Class ;
51         rdfs:subClassOf [ rdf:type owl:Restriction ;
52                 owl:onProperty :properPartOf ;
53                 owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ;
54                 owl:onClass :ClassA
55               ] .
56
57 ###  http://www.umlintoowlex.org/semicex#ClassD
58 :ClassD rdf:type owl:Class ;
59         rdfs:subClassOf :ClassB .
```

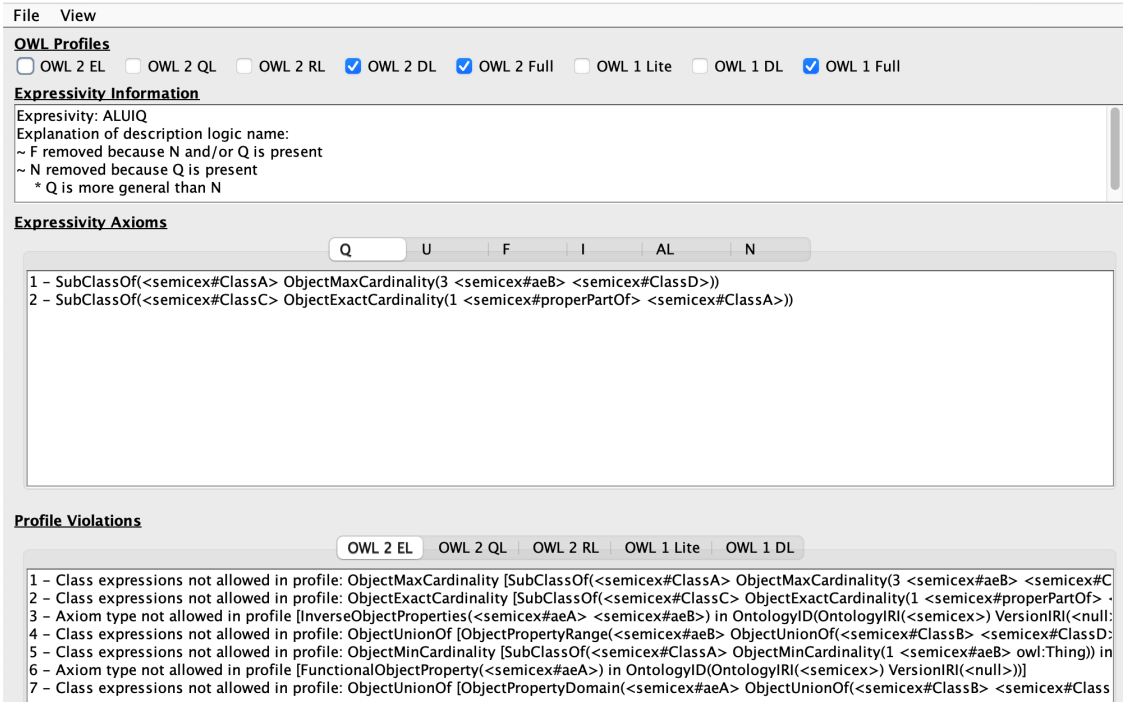

Figure 2.5: DL fragment and OWL species membership for the 'conceptual data modelling' style of UML class diagram model conversion of the diagram in Figure 2.3 and in Listing 2.2 (RDF/XML serialisation used), as in the OWL Classifier v1.3.

2.1.2 Out of scope features

Several efforts are underway that cover adjacent comparisons and benchmarking, or topics that could have been elaborated on but are not in this comparison. The complementary tool-based *comparison at end-user tool level* covered features such as whether it generates HTML, whether it is stand-alone or web-based and so on, and the reader is referred to that report for those details; the outcome table is included in Appendix B for easy access.

Also out of scope are features about the *processes and procedures* of creating artefacts and *auxiliary artefacts* that are not RDF, RDFS, or OWL, such as documentation in HTML, JSON-LD specifics, and SHACL shapes, and how exactly they differ across the tools. Some tools also can import and export files in other formats, such as MS excel or csv, which is deemed out of scope as well.

There are other features that were included but not to the detail they could have been in hindsight, or they were upfront already relegated to a different project. For instance, it was unknown whether many tools would have an internal metamodel as central pivot—such as the one most explicitly documented for crowd2 [14] in [34]—and therefore only its presence or absence is recorded, not also a comparison of the respective metamodel's content and adequacy. Likewise, only presence or absence of the features to record metadata are recorded, and if a public model for metadata, which one, not the respective model's content to record metadata and how much metadata and in what format.

2.2 Materials

This section consists principally of brief tool descriptions and brief notes on pertinent standards and specifications. The artefacts created for each tool were eventually not specified upfront, and only created or used as provided per tool, because of the wide variety of languages and tool features, which makes a strict use case-based approach less effective. Of each tool, it was attempted to create a specification that used all advertised language features.

2.2.1 Tool descriptions

The tools are grouped by their scope. Yhteentoimivuus, Dataspecer, and VocBench are grouped in the 'RDF and OWL tools', INSPEC is a set of rules for tools for the 'RDF and OWL tools' approach, model2owl, the SEMIC toolchain, and uml2semantics all convert UML class diagrams, and LinkML is a programming environment toolset that also has support for RDF and OWL.

Yhteentoimivuus

The Yhteentoimivuus (En.: Compatibility) emanated from a Finnish interoperability project, which aimed to be the national application and implementation of the European Interoperability Framework (EIF)[5]. It allows users to create data vocabularies, models, and other artefacts through a Web interface. The project itself concluded in 2018, but the software is maintained and the tool is in use. The software is open source and the code is available at `https://github.com/VRK-YTI`.

VocBench

VocBench v3 [49] uses Semantic Turkey [47] as back-end and is downloadable from the Web[6]. It has its origins in SKOS vocabulary development and AGROVOC and in serving multilingual communities. Support for OWL has been added gradually over the years, albeit not the standard DL reasoning services. VocBench offers a web-based GUI/front-end and there are various installations[7]. For this evaluation, we eventually installed the latest version (v14.0.0) locally.

Dataspecer

Dataspecer [50] focuses on management of semantic data specifications, including vocabularies and application profiles as well as derived artefacts, such as physical (/data/implementation) schemas and validation rules. There are a number of scientific papers describing the design rationale and its various components [38, 50]. Dataspecer is available for use in

[5] See `https://wiki.dvv.fi/spaces/YTIJD/pages/34742731/Finnish+interoperability+platform+and+tools+-+introduction`; last accessed in February 2026

[6] `https://vocbench.uniroma2.it/doc/` and `https://semanticturkey.uniroma2.it/`, respectively.

[7] `https://vocbench.uniroma2.it/`, `https://vocbench.dfc-standard.org/vocbench3/#/Home`, whereas `https://vocbench.op.europa.eu/` is only for EU institution.

a Web browser[8], including a tutorial. Its code is available on GitHub under a MIT licence[9] where also details are documented. A number of sample specifications are available from `https://mff-uk.github.io/specifications/`.

INSPEC

INSPEC, or: Interoperable Specifications [19], consists of independent specifications of rules for developing various models, currently covering terminologies, data vocabularies, and application profiles [7]. It is not a tool, and it aims to be tool-agnostic to the extent that any tool that adheres to the rules is deemed acceptable. As such, it is better suited to be compared to modelling principles of the OBO Foundry and the SEMIC Style Guide UML convention rules.

To concretise and compare it to other tools included in this comparison, we selected the open source platform EntryScape[10], which is also used to host the Swedish Data portal that uses the INSPEC rules.

SEMIC toolchain

The SEMIC toolchain[11] is used for generating CVs and APs and is based on the OSLO toolchain v3[12]. A new version (v4) of the OSLO toolchain is available, but it is not yet deployed as SEMIC Toolchain. Version 3 takes EA v15 eap files as input; version 4 takes v16 as input. It maps native EA format into its internal metamodel and then generates artefacts based on the selected 'model type': for a CV, it will generate a Turtle file, and for an AP, it will generate JSON-LD and SHACL shapes.[13] The publication workflow/toolchain is built around Git repos with CI/CD and Docker images.

model2owl

Model2owl [1] was developed for the Publications office of the EU. It takes as input UML XMI 2.5.1 and 2.5 exported from EA and outputs Turtle and plain RDF in two versions ("core" and "restrictions"), SHACL shapes, JSON-LD, and a human-readable HTML specification. There is publicly available documentation about the generation [2]. Multiple versions of model2owl exists, which do not all have the same features. Upon inquiry with the lead developer, its release/3.2.0 was used[14]. Its main production-level usage is for the development of the eProcurement Ontology.

[8] `https://dataspecer.com/`

[9] `https://github.com/dataspecer/dataspecer`

[10] `https://entryscape.org/`

[11] `https://semiceu.github.io/toolchain-manual/`

[12] `https://github.com/Informatievlaanderen/OSLO-UML-Transformer`

[13] `https://github.com/Informatievlaanderen/OSLO-UML-Transformer/wiki`; last accessed on 26 March 2026.

[14] `https://github.com/OP-TED/model2owl/tree/release/3.2.0`

uml2semantics

The uml2semantics tool[15] was developed by Henriette Harmse from the European Bioinformatics Institute (EBI). It takes as input custom TSV or EA or both and outputs OWL (RDF/XML). It is an early-stage tool and production-level use is unclear.

LinkML

LinkML[16] [43] is not strictly within the SEMIC scope at present, but enjoys uptake in the Linked Data space, supports converting YAML to RDF with the rdflib API, and has an OWL generator for converting YAML to OWL, and therefore was included as well. It does not consume UML and only can produce UML from a LinkML model as YAML file.

2.2.2 Auxiliary materials

Pertinent standards and specifications include the following ones:

- The *Web Ontology Language Standard OWL 2* [42] is intended for representing and reasoning over knowledge, but also has language features such that it can be used to formalise semantic data specifications (information/conceptual data models), notably data properties and data types. It has five alternative syntaxes, of which the RDF/XML is mandatory and the others are optional. OWL 2 consists of 4 DL-based languages (OWL 2 DL, EL, QL, RL), which are all within the decidable fragment of first order logic, and one RDF-based one (OWL 2 Full), which is undecidable. See Appendix A for a summary of features. Reasoning operates under the Open World Assumption (OWA) and OWL does not have the unique name assumption (nUNA). The standard reasoning services are consistency of the ontology, class and object property satisfiability, class and object property subsumption, instance checking (is a (resp. (a, b)) a member of class C (resp. R)), query answering/instance retrieval (find all members of C in $\mathcal{KB}$).
- The *Shapes Constraint Language SHACL* [37] is intended for validating RDF graphs against a set of conditions represented as shapes graphs. Validation should happen under the Closed World Assumption (CWA) that can be further restricted with `sh:closed`, and SHACL operates under the unique name assumption (UNA). As such, it is more aligned with established assumptions in conceptual data modelling, databases, and object-oriented application software behaviour than OWL with its automated reasoning services.
- *Resource Description Framework RDF* [17] is intended for representing data and information on the Web and is represented as a set of triples that form a graph, which can be serialised in a number of syntaxes, such as Turtle and JSON-LD. RDF can be used as a data structure for storing content intended as a graph and it can also be used as a serialisation language for other languages (e.g., OWL, RDFS, SHACL, and SKOS). Where it used to be common to use XML as serialisation language, this is gradually being replaced by serialisations in RDF.

[15] `https://github.com/henriettеharmse/uml2semantics`
[16] `https://linkml.io/`

- *RDF Schema* [15] “provides a data-modelling vocabulary for RDF data”, extending plain RDF to describe “groups of related resources and the relationships between these resources”, or, informally: to classes and the properties between them. See Appendix A for a summary of features.
- The *Simple Knowledge Organisation System SKOS* [30] was developed to try to get terminologists and thesauri and vocabulary developers on board with the W3C and Semantic Web technologies and to convert their artefacts into the same syntax. This is also reflected in the primitives of the language, such are ‘broader’, ‘narrower’, and ‘preferred label’. An unfortunate mapping of SKOS into OWL converts the vocabulary concepts into individuals whereas they are typically intended to be or ‘thought of as’ concepts, not instances of the class concept.
- The *ontolex-lemon community report* [16] offers a model for multilingual ontologies, ontology localisation and internationalisation, which could be relevant for uptake of CVs and APs across countries.
- The suite of relevant standards from the Object Management Group, being the *Unified Modeling Language UML 2.5.1* [45] for UML class diagrams and the *XML Metadata Interchange XMI 2.5.1* [46] for the serialisation of UML models in XML.

Auxiliary tools used concern syntax checking and conversions with Protégé Desktop [25] v5.6.5, the OWL species and DL logic assessments with the OWL Classifier v1.3[17], and spreadsheets to manage the inventarisation of tools and the comparison.

[17] `https://github.com/muhummadPatel/OWL_Classifier`; last accessed 3 April 2026.

3 Results

A total of 27 tools and related resources were harvested and categorised into ‘RDF, OWL or the like input’ (n=5), ‘UML to OWL’ (n=16), ‘UML directly into OWL during the modelling activity’ (n=2), ‘relevant paper-based efforts on UML to OWL’ (n=2), and ‘possibly relevant but neither UML nor OWL as input’ (n=3). Nine were selected for the comparison, as described in Section 2.2.1; the other tools are listed in Appendix C.

3.1 Features compared

The comparison output is summarised in the tables included in this section, where values marked with an asterisk have qualifying comments in the respective subsection in Section 3.2. In all cases, a multi-modal approach to information gathering was used, combining documentation, tool experimentation, inspection of available and created models, and email and/or an online meeting with a lead developer, although not all of them for all tools.

The current Style Guide focuses on UML, and therefore we will commence with it. Coverage of the UML and UML-like features are included in Table 3.1, which excludes VocBench because it does not have a graphical editor and INSPEC because it is not a tool (hence, all ‘N/A’ for both). Dataspecer and Yhteentoimivuus are included, though note that they do not intend to follow UML notation nor intend to support it, but merely have a graphical modelling option for the language features they allow modellers to use, and the notation bears some resemblance to UML class diagram notation, and therefore should be read in that manner. The UML class diagram serialisation question (Table 3.2) is applied only to the three UML class diagram formalisation tools.

The comparison of how model elements are formalised or serialised is included in Table 3.3, where the numbers in the first column refer to the items listed in Section 2.1.1 as well as what the values mean precisely, among the reasonable alternatives. This is followed by the assessment of what tool developers mean with ‘RDF’ and/or ‘OWL’ exactly and certain related modelling patterns (Table 3.4). Finally, the comparison of the selection of attendant tool features is shown in Table 3.5. (Any mistake in the tables is the authors’ or the spreadsheet’s seemingly not very reliable drop-down cell value validation feature.)

Table 3.1: Coverage of the UML and UML-like features in the tool; *: see comments in the text.

UML feature	**Icons equivalent to UML**		**UML features supported in transformation**		
	Dataspecer *	*Yhteentoimivuus* *	*model2owl*	*SEMIC Toolchain*	*uml2semantics*
Association	binary	binary	binary	binary	binary
Association end	N/A	N/A	single allowed	single allowed	single allowed
Aggregation, shared composite	none	none	none	both	none
Navigability	no	yes	yes	yes	no
Dependency	no	no	yes	yes	no
Multiplicity: basic (0..*, 0..1, 1..1, 1..*) on association or attribute	on both *	on both	on both	on both	on both
Multiplicity: arbitrary numbers, on associations & attributes	no	no	yes, both	yes, both	yes, both
UML datatypes	N/A	N/A	built-in, XSD, RDF	XSD, user-defined domains	XSD, user-defined domains
Generalisation on classes, associations	both	classes	both	classes, association end	both
Disjoint	no	classes	classes	no	classes
Covering	no	no	no	no	yes
Qualified association	no	no	no	no	no
Association class	no	no	no	yes	no
Value specification	no	no	no	no	no
Attribute value constraint	no	no	yes	no	no
Stereotypes for types	neither	neither	on class	on class	on datatype
Stereotypes for 'modality' of a multiplicity	association, attribute	neither	ignored	ignored	ignored
Textual definitions, notes	yes	yes	yes	yes	yes
User-defined tags	no	N/A	yes	yes	yes
Model metadata	no	yes	yes	yes	yes
Optional {id} construct (mandatory 1:1)	no	no	no	no	no

Table 3.2: UML diagram serialisation files used by the UML to RDF tool; *: see comments in the text.

Feature	**model2owl**	**SEMIC Toolchain**	**uml2semantics**
Core XMI for UML is used for main elements	yes	no	no, optional
Main UML features also in product-specific xmi:Extension or extension	yes (tagged values, stereotypes)	yes (tagged values)	N/A
XMI version	2.5 or higher	N/A	N/A
Tool that generates XMI that's inputted	EA	N/A	N/A
Tool-specific serialisation (non-XMI)	N/A	eap format (SQLite)	custom TSV or EA 17.1 *

Table 3.3: Comparison of the formalisation of the content in the interface or model into RDF, RDFS, or OWL. The numbers refer to the items listed in Section 2.1.1. Abbreviations: Dataspec.: Dataspecer; Yht: Yhteentoimivuus; SEMIC TC: SEMIC Toolchain; u2s: uml2semantics; LML owl g.: LinkML OWL generator; OP: object property; DP: data property; D&R: domain & range; *: see comments in the text.

Feature		**Diagram/form/interface to OWL**			**UML to OWL**			**YAMLtoOWL**
	INSPEC	*Dataspec.*	*Yht.*	*VocBench*	*model2owl*	*SEMIC TC*	*u2s*	*LML owl g.*
Element names or labels + IDs (1)	both possible	element names	both possible	element names	element names	element names	element names	element names
One or both association ends bumped up to OP (2)	N/A	N/A	N/A	N/A	yes, either	yes, either *	yes, either	N/A
Association ends bumped up to OP, declared inverses (3)	N/A	N/A	N/A	N/A	yes	no	yes	N/A
Association to OP (4)	yes	yes	yes	N/A	no	yes *	yes	yes
Aggregation conversion (5, 6)	N/A	N/A	N/A	N/A	error	plain OP (with po/hp or ppo/hpp name) *	ignored	N/A
Navigability (7)	N/A	N/A	N/A	N/A	approximated with OP in one direction	ignored	ignored	N/A
Dependency (8)	N/A	N/A	N/A	N/A	approximated with OP in one direction	approximated with OP in one direction	ignored	N/A
Domain and range axiom for each OP/association/association end (9)	N/A	yes	no	N/A	yes	yes	yes	partially
Association/association end to OP with class expression axiom (10)	no	no	yes	N/A	no	no	yes	N/A
Multiplicity to qualified cardinality (11)	N/A	no	N/A	N/A	yes	N/A	yes	no
Multiplicity to unqualified card. + D&R axioms (12)	no	yes *	N/A	N/A	no	yes	no	yes
N-aries where n>2 approximated or ignored (13)	N/A	N/A	N/A	approximated	error *	approximated	error	unknown

Feature	*INSPEC*	*Dataspec.*	*Yht.*	*VocBench*	*model2owl*	*SEMIC TC*	*u2s*	*LML owl g.*
OP/DP reuse (15)	unknown	yes but intersection instead of union	new OP/DPs created w diff names	N/A	union of classes in domain or range	union of classes in domain or range	union of classes in domain or range *	yes with CE, no/limited D&R
Attribute to DP with class/domain, DT/range (14)	unknown	domain & range specific	domain & range specific	N/A	domain & range specific	domain & range specific	domain & range specific	domain & range specific
Existential quantification (1..* and 1..1 in UML diagram) (16)	N/A	N/A *	\exists	either	min 1	min 1	min 1	min 1 *
Exactly 1 formalisation (17)	N/A	N/A *	N/A	exactly 1	exactly 1	min+max	exactly 1	min 1 + max 1
Universal quantification (0..* or 0..1 in UML diagram) (18)	N/A	D/R axiom	N/A	either	D/R axiom	D/R axiom	D/R axiom	min 0
Classes outside the hierarchy are declared disjoint (19)	N/A	no	no	no	no *	no	no	no
Stereotypes for types (20)	N/A	N/A	N/A	N/A	other *	not supported (ignored) *	other *	N/A
stereotyes for modalities (21)	N/A	N/A	N/A	N/A	ignored	ignored *	ignored	N/A
Uses a foundational ontology (22)	no	possible *	no	possible	no	no	no	no
advanced language features (23)	none	none	property chain	OP characteristics *	none	none	one-of	none
Abuse of notation in conventions for UML class diagrams (24)	N/A	N/A	N/A	N/A	yes *	no	no *	N/A
Requires adherence to UML class diagram modelling conventions for the transformation to work (25)	no	no	no	N/A	yes	yes	no	N/A
Tweaks to UML constructs in modelling conventions (26)	N/A	N/A	N/A	N/A	yes (changes semantics) *	no *	no	N/A

Table 3.4: The RDF, OWL, and DL species generated by the tools and any modelling patterns. Abbreviations: Dataspec.: Dataspecer; Yht: Yhteentoimivuus; SEMIC TC: SEMIC Toolchain; u2s: uml2semantics; LML owl g.: LinkML OWL generator; *: see comments in the text.

Feature		**Diagram/form/interface to OWL**			**UML to OWL**			**YAMLtoOWL**
	INSPEC	*Dataspec.*	*Yht.*	*VocBench*	*model2owl*	*SEMIC TC*	*u2s*	*LML owl g.*
Lives in RDF 'world', files are invalid OWL or they are not guaranteed to be	yes	yes *	yes	yes *	yes *	yes *	no	yes
The 'RDF' is actually RDFS (data schema, no vocab or ontology layer)	yes	no	no	no	no	no	no	no
The 'RDF' is the Turtle serialisation format of OWL, and so compliant with OWL standard	N/A	no	no	no *	no	no	yes	no
As previous + within the DL fragment (i.e., no accidental meddling with the language and the files aren't in OWL (2) Full but in a suitable DL fragment)	N/A	yes	no	no	no	no	yes	no
The 'RDF' is a specific (constrained flavour of) another model	yes *	yes *	depends *	depends *	no	no	no	yes
OWL species supported	OWL 2 Full	OWL 2 Full *	OWL 2 EL *	OWL 2 DL *	OWL DL, syntax issue *	N/A *	OWL 2 DL	OWL 2 Full *
Any declarations pushing it into OWL Full or OWL 2 Full are metadata mistakes	N/A	no	no	no	N/A *	yes *	no	no
DL fragment supported	$\mathcal{ALH}(D)$ *	$\mathcal{ALH}(D)$ at least *	$\mathcal{SRF}(D)$	$\mathcal{SRIQ}$ *	unknown *	$\mathcal{AL}(D)$ for the CVs	a fragment of $\mathcal{SROIQ}(D)$ *	at least $\mathcal{ALCN}(D)$ *
Output format	N/A	Turtle	Turtle, OWL/XML	Turtle,Manchester *	Turtle, RDF	Turtle, RDF	RDF/XML	Turtle
Input format if OWL (see also "Disambiguation of 'RDF' below")	N/A	N/A	N/A	Turtle	N/A	N/A	N/A	N/A
Can convert OWL format in same tool	N/A	no	no	yes	no	no	no	no
Metadata converted from input file	N/A	no	N/A	yes	yes	yes	N/A	yes
Element annotations converted	N/A	yes	N/A	yes	yes	yes	yes	yes
Uses SKOS where native OWL can	yes	no	depends *	no	yes	yes	no	yes

Feature	*INSPEC*	*Dataspec.*	*Yht.*	*VocBench*	*model2owl*	*SEMIC TC*	*u2s*	*LML owl g.*
Modelling conventions described	yes *	no	yes	no	yes	yes *	yes	no
Naming scheme recommended	none	CamelCase	none	none	CamelCase	none	none	no
Uses pre-defined ODPs (as ODP or as templates/forms to fill in) to guide usage	no	yes *	yes *	no	no	no	no	no
Built-in dependencies with other files or formats	yes	yes *	yes	no	yes *	yes	no	yes *
Existential quantification	N/A	N/A *	\exists	\exists	min 1	N/A	min 1	\exists *
Domain and range axiom for each OP/assoc/assoc end	no	yes	yes	no	yes	configurable	yes	yes
Inverses or inverse()	neither	neither	neither	inverses	inverses	neither	inverses *	neither
Qualified number restrictions	N/A	N/A	N/A	OWL 2 qualified *	OWL 2 qualified	N/A	OWL 2 qualified	OWL DL/lite unqualified

Table 3.5: Selected tool features. Abbreviations: Dataspec.: Dataspecer; Yht: Yhteentoimivuus; SEMIC TC: SEMIC Toolchain; u2s: uml2semantics; LML owl g.: LinkML OWL generator; *: see comments in the text.

Feature		Diagram/form/interface to OWL			UML to OWL			YAMLtoOWL
	INSPEC	*Dataspec.*	*Yht.*	*VocBench*	*model2owl*	*SEMIC TC*	*u2s*	*LML owl g.*
Metadata can be declared	yes	no *	yes	yes	yes	yes	no	configurable *
Metadata declared using SRM	no	N/A	no *	no	no	yes	N/A	no
API usage	N/A	other	unknown *	RDF4J API *	other *	other *	OWL API	unknown
OWL species can be chosen for the transformation	N/A	no	no	N/A	no	no	no	no
Formalisation approach	mappings, algorithmic/ rules	mappings *	mappings	N/A	algorithmic/ rules	algorithmic/ rules	algorithmic/ rules	algorithmic/ rules
Execution	N/A	on-the-go	on-the-go	on-the-go	batch	batch	batch	batch
Uses an internal metamodel	no	yes *	partially	no	no	yes *	no *	yes

Feature	*INSPEC*	*Dataspec.*	*Yht.*	*VocBench*	*model2owl*	*SEMIC TC*	*u2s*	*LML owl g.*
Takes multiple input formats (e..g, different XMI versions, XMI from different tools, RDF/XML and Turtle)	N/A	no	yes	no	no	no	yes	no
Formalisation options configurable	no	no	no	no	no	yes *	no	yes *
Constraints/'conventions' on UML diagram and checking for it	N/A	N/A	N/A	N/A	yes *	yes *	no	N/A
Built-in reasoner	N/A	no	no	no *	no	no	no	no
Imports of other artefacts (ontologies, vocabs)	hard (full), soft *	hard (full)	soft, hard (full)	hard (full, module), soft *	hard (full), soft	soft	no	soft *
Generates syntactically correct OWL and generated file can be processed by other tools	N/A	no (tool) *	partial *	no (tool) *	no (tool, external) *	no (tool)	partial *	partial *
Visual modelling (i.e., editing and not just viewing), with 'static'/stable diagrams	no	yes	yes	no	no	no	no	no *
Converts (UML) constraints to SHACL shapes	N/A	yes (in addition to OWL)	yes (in addition to OWL)	N/A	yes (in addition to OWL)	yes (instead of OWL) *	no	no
Output file format in OWL but file is *.ttl *.rdf etc instead of *.owl	N/A	ttl, may not be valid owl	ttl and rdf, may not be valid owl	ttl and rdf, may not be valid owl *	ttl and rdf, may not be valid owl owl	ttl and rdf, may not be valid owl	no	ttl, may not be valid owl or in OWL full

3.2 Clarifications by tool

Qualifying notes apply to each tool on what was done and how, and to the starred values in the tables from the previous section, which are briefly described next, and largely in note form only. *This section can safely be skipped if the results in the table are clear in its entirety or for the parts that the reader is interested in.*

The RDF and OWL tools are discussed first, followed by the UML-based tools.

3.2.1 VocBench notes

The notes pertain mainly to the observations from testing the tool. The usage setting tested first was OWL + RDFS project (other options are: SKOS, SKOSXL, OntoLex) and the 'trivial inference' checkbox was selected. While VocBench claims support for RDFS, OWL 2 RL and OWL 2 QL specifically [49] (pp13-14), we managed to create an ontology in OWL 2 DL and the $\mathcal{SRIQ}$ fragment specifically, through asserting a class expression with qualified cardinality, and irreflexivity and transitivity on object properties, but only when OWL is selected for the project upfront and great care is taken when adding knowledge.

Issues were observed with OWL imports, which may be due to the RDF/XML serialisation. VocBench did not let us import a `.owl` file, but it presumably should be possible to use after either converting an ontology into Turtle format or at least change the file extension to `.ttl`. When selecting an RDFS + RDFS type of project, the import feature worked (attempted with `m8g.ttl`). There, the interface also allowed asserting a qualified cardinality constraint, which, however, was converted in a `SomeValuesFrom` in the serialisation, i.e., logically different and erroneous with respect to the GUI. Qualified cardinality is not a feature of RDFS, nor is existential quantification.

Regarding formats, this depends on the type of project chosen. For OWL, on "The 'RDF' is the Turtle serialisation format of OWL, and so compliant with OWL standard" (Table 3.4), it was answered in the negative, because there are too many ways to create invalid code that is not flagged, and the default serialisation is in RDF rather than Turtle or the compact RDF/XML. VocBench can generate Turtle syntax, among many formats, when exporting the file. In that configuration, it does not abuse SKOS where native OWL features can be used. Conversely, for a SKOS project, the 'RDF' is a specific (constrained flavour of) another model and is indeed SKOS, as assessed by quick manual inspection.

The interface invites asserting things that are guaranteed to result in incorrect syntax, such as adding a property `owl:someValuesFrom` without an actual property and using the wrong language features (one can pick from many languages in the drop-down), therefore so the generated code is practically very likely to be syntactically incorrect, be it OWL or another format. For any recommended use, as a minimum, the interface should be updated to ensure only syntactically correct files will be generated, rather than being drown in drop-own options that result in syntactically incorrect files.

VocBench uses the RDF4J API[1] that does not claim OWL compliance, and VocBench does not carry out any syntax checking. It also does not offer standard DL reasoning services such as consistency and satisfiability checking. Upon project creation, one can select "trivial

[1] `https://rdf4j.org/`

inferences", which we did, but it had no effect.

Many output formats are listed regardless the project type chosen, such as NDJSON-LD, N-quads, and TriX , which is considered out of scope for this comparison.

3.2.2 Dataspecer notes

The demo instance was used for the practical assessment, which was confirmed to have the same functionality as the regular tool. Dataspecer has an editor interface with a visual notation that closely resembles UML, and therefore the UML feature section was filled in in Table 3.1, though note that it does not aim for UML compliance.

It has the key design decision of separating the original from reuse, in that one section deals with the permissive core specifications at the type-level, such as the vocabularies and lightweight ontologies, and in another section are the application profiles that are treated as instances of the permissive core specifications. This also means that an AP, such as DCAT-AP-CZ is serialised at the instance-level, not at the type-level that the SEMIC toolchain, model2owl, VocBench, and uml2semantics do.

Other consequences of this design decision are that multiplicity can only be set in APs once the classes are also 'instantiated' in the AP, not in vocabularies. Such constraints are stored in its tailored metamodel but they do not function as constraints at the logical level in that serialisation.[2] Features concerning cardinality constraints are therefore set to N/A. In APs, the only options are 0, 1, and *, not arbitrary numbers. Also for APs, one can select optional/recommended/mandatory, and while not strictly speaking stereotypes, *de facto* they are treated as such and therefore the value was set to 'yes'.

For each "relationship", domain and range classes have to be added (item 9). In case of property reuse, the classes end up as intersection rather than union of item 15b for the data property it was checked with.

Adding a new relationship can be done with 'source' and 'target', although it is drawn as a relationship with a line and arrow alike UML's navigability. Since it does not claim UML, the corresponding feature's value was set on N/A despite 'misuse' of the navigability notation and source/target terminology.

The tool uses the Data Specification Vocabulary (DSV) [36] as part of an explicit internal metamodel, which is the metamodel for the elements that can appear in Dataspecer, and it also avails of PROF [9]. It does not include features for adding metadata, which is under development[3].

Regarding the DL fragment, it is along the line of $\mathcal{ALH}(D)$ when developing a vocabulary, but it ends up in OWL 2 Full due to OWL syntax violations because of the use of reserved vocabulary for class IRIs, as a knock-on effect from the DSV model that does so (e.g., rdfs:Class explicitly as a class in the TBox). The documentation suggests, and the lead developer confirmed (Jakub Klímek (pers. comm.), that (the designer of) Dataspecer does not aim to support ontologies or OWL. The same front page does claim "Use your domain ontology or vocabulary to semi-automatically derive semantically compliant data schemas

[2] For SHACL shapes generation, they are converted into their respective `minCount` and `maxCount`, reading in the declarative part and converting them into actual constraints

[3] `https://github.com/dataspecer/dataspecer/issues/1212`

for different data formats such as XML, JSON and CSV and let Dataspecer guide you in the process, fostering technical data interoperability." and the tool allows saving as "RDFS/OWL" that results in a `*.ttl` file that can be read by both Protégé 5.6.5 and the more strict OWL Classifier. More precisely: the file can be processed but has issues. The APs provided[4] are indeed pushed into the instance-level rather than the class-level.

The formalisation approach uses mappings, given the internal metamodel. While the formalisation options are not configurable upon formalisation/serialisation, it is possible to choose 'profiles', or types of models, to create, being: limited to vocabulary, application profile, and data model. The interface is adjusted accordingly through forms, together with what features are permitted to be used, which is akin to an ODP.

3.2.3 Yhteentoimivuus notes

Both the freely accessible read access version of Yhteentoimivuus was used in the comparison assessment (when logged in to Eduuni) and the edit access using a temporary SEMICtest organisation, given that edit access is organisation-dependent, with thanks to the Project lead Riita Alkula[5]. The answers are based on the tool access, the extensive emailed documentation, and the online material[6].

The tool has multiple entry points for the different types of models, being terminologies/-controlled vocabularies, core vocabularies, application profiles, and physical data models. The read-access option shows a basic interface and list of features[7], as do the UML-lookalike diagrams. Since it has a visual notation, the UML features were filled in after all, noting that UML support was not intended by the developers and it should not be read as equivalent feature availability. For instance, the data types are not the UML datatypes, but mostly XSD data types and a few others, such as `rdfs:Literal`. Also, multiplicity constraints depend on the type of model one builds; e.g., for data vocabularies, none can be declared explicitly, but it is formalised with 1..*.

Yhteentoimivuus has support for RDFS and OWL, according to the emailed documentation, and OWL 2 EL specifically, as well as several other formats for import and export, such as importing from csv and generating SHACL shapes (SHACL core only). The data vocabularies presentation[8], states that "Internally, metadata descriptions are based on RDF, SKOS, OWL, and SHACL. From internal descriptions, various output formats (serialisations) can be generated, including RDF, JSON-LD, Open API, Turtle, and XML formats." and the documentation of the architecture states SHACL, XML, and JSON for the data models, and

[4] Available through `https://mff-uk.github.io/specifications/` and tested with DCAT-AP v 3.0.1 and DCAT-AP-CZ.

[5] Editing is possible after an account upgrade, which is permitted for employees from a pre-defined list of Finnish organisations.

[6] Available via `https://wiki.dvv.fi/spaces/YTIJD/pages/27269153/Material+in+English`

[7] `https://kehittajille.suomi.fi/services/finnish-interoperability-platform/instructions-for-the-data-vocabularies-tool/creation-publication-and-versioning-of-a-datamodel/creation-of-a-new-datamodel`

[8] Available at `https://wiki.dvv.fi/spaces/YTIJD/pages/27269153/Material+in+English`; last accessed d.d. 9 February 2026.

SKOS for terminologies[9]. These internal descriptions are specific for each type of model[10], not one metamodel for all types of models.

For the initial probing of tool behaviour, three vocabularies were randomly selected for download in the formats of interest (RDF and Turtle), being the Accessibility Glossary, the Terminology of Built Environment, and the DigiOne data model. The first two were principally SKOS and SKOS-XL, even though the file extensions and serialisation are as `.rdf` and `.ttl` output. The 'RDF' is thus a specific (constrained flavour of) another model, which gave rise to answering 'yes' to the feature "Uses SKOS where native OWL can do the job" and for "The 'RDF' is a specific (constrained flavour of) another model" (Table 3.4), but when creating a data vocabulary model, it uses by default `dct:terms` and `suomi-meta`; therefore, an "it depends" is the more precise answer. The files inspected did not demonstrate that fully, with SKOS in Turtle still being SKOS, and their conversion carried out 1:1, in that a SKOS concept becomes a class in OWL and the concepts in SKOS turn into instances. Further, the DigiOne data model contains a non-OWL part with many `dcterms:hasPart` declarations, other lines starting with "`**`" or "`-`" within the `owl:Ontology` tags, followed by the OWL/XML serialisation outside it. It does not load in other tools, not even in the lenient/second-guessing Protégé 5.6.5.

While investigating issued for debugging is out-of-scope, we did create a model in the tool to exclude the option that it may have been the end user who created file issues. The interface allows for, and serialises, transitivity, class disjointness, and functional data properties, which is indeed within OWL 2 EL and the generated files are in the DL $\mathcal{SRF}(D)$ specifically, but it serialises in OWL 2 Full if either no domain and range are declared for a property, or no target is specified in what is to become a class expression. It also does not allow property reuse, causing the error message of "not enough operands", which pushes it into OWL 2 Full rather than OWL 2 EL. The tool itself does not check whether the exported file is syntactically correct. Therefore, "partial" was selected for whether it generates syntactically correct files. If domains and ranges and "target" (role filler) are defined explicitly, then the output file is within OWL 2 EL.

Two observations are that the collection of properties, presumably subclass expression, are formalised with `EquivalentClasses` rather than `SubClassOf`. Note: in the interface, "Equivalent Classes" is used for atomic classes only. This may lead to unintended deductions, as will the forced domain and range axioms likely result in unintended deductions unless the classes are sufficiently high up in the hierarchy.

Based on the tool screenshots, there appear to be *de facto* ODPs, in the form of the fields to fill in, but the edit interface was permissive. Practically, the edit interface constitutes an ODP, in that it serialises as a 'defined concept/class with all-some' for the properties.

The metadata schema is specific to the tool, with online documentation[11], and thus amounts to the built-in dependencies with other files or formats. It also uses dcterms for at least the data vocabularies. Yhteentoimivuus' metamodel schema appears to be more

[9] a discrepancy with the diagram at `https://wiki.dvv.fi/download/attachments/27269153/Information%20Architecture%20of%20the%20Interoperability%20Platform.png?api=v2`; last accessed at 9 February 2026.

[10] e.g.: `https://tietomallit.suomi.fi/en/model/term?ver=0.1.0`; last accessed 19 March 2026.

[11] Accessible at `https://cscfi.github.io/mscr-docs/mscr/schema-view/`; last accessed on 10 March 2026.

encompassing than SRM so it would be able to accommodate it (Alkula, pers. comm.)

The API usage for the serialisations and formalisation approach were asked about, but could not be ascertained within the allotted timeframe, and therefore were set to 'unknown'. API usage mentioned in the documentation refers to API endpoints for one model to access content from another one.

3.2.4 INSPEC and EntryScape notes

The data was gathered from all documentation available on the INSPEC website [19], the related GitHub repository[12], the presentation at the SEMIC2025 conference, a demo instance of EntryScape, and an online meeting.

Interoperable specifications

INSPEC consists of a set of rules for interoperable specifications (based on PROF [9]), for data vocabularies that are based on RDFS [15], for terminologies use SKOS [30], for application profiles in SHACL [37] and for diagrams in SVG. The 'RDF' use is a constrained flavour, be it PROF [9], or what is, or should be, de facto an OWL file restricted to what can be expressed in the RDFS fragment (see Appendix A).

For RDFS, when that is indeed limited to only classes, properties, class and property hierarchies, and domain and range restrictions, then it is within the expressiveness of all OWL species, and the DL $\mathcal{ALH}(D)$ at most.

INSPEC's "data vocabularies" corresponds in intent to SEMIC CVs. It operationalises the design decision that *all* constraints and *any* reuse is relegated to the AP layer and it then appears only in the SHACL shapes, not the RDFS document. Its rule DV-2 requires that the RDFS document must be an OWL file.

Interpretation of the rules requires context. For instance, for APs, "Rule AP-1: An application profile MUST be expressed in a single RDF Dataset": this is not to be understood as APs being inherently instances (since 'dataset'), but that it is serialised in SHACL, and the syntax of SHACL is in RDF.

Several modelling conventions are described online[13], such as URIs for elements (DV-4), that a node shape may express that it refines another node shape provided it uses `inspec:refines` (AP-9), and that all classes and properties in the vocabulary must point to the data vocabulary resource using `rdfs:isDefinedBy` (DV-3). There are no conventions about format of the names of the elements, whether properties can be reused in one specification, or whether domain and ranges have to be declared.

Imports of other artefacts are stated as permitted in the SHACL shapes file, using the `owl:imports` statement (rule AP-13), and through soft import and subclassing in the vocabulary (DV-5). The latter also relates to some confusion regarding subclassing and reuse[14], whose discussion is beyond the current scope of fact-finding and comparison.

[12] `https://github.com/diggsweden/interoperable-specifications`; last accessed on 12 March 2026.

[13] Available at `https://diggsweden.github.io/interoperable-specifications/docs/rules.html` and described here: `https://diggsweden.github.io/interoperable-specifications/docs/`

[14] elaborated on here: `https://diggsweden.github.io/interoperable-specifications/docs/subclassing.html`

Visual modelling is optional, but it is expected that an SVG file is generated that visualises the specification.

EntryScape

Since INSPEC is intended to be tool-agnostic, multiple tools may merit inclusion in the comparison. This was carried out in part for one such tool, EntryScape, but not close to being completed due to time constraints and therefore it is excluded from this report.

3.2.5 model2owl notes

Both the documentation and the tool was used for the comparison, noting that the published documentation on the TED website is out of sync with the tool's feature set. Where it differs, the implementation was used and the differences are mentioned in this section.

Regarding 'classes outside the hierarchy are declared disjoint' (item 19): they are not. Moreover, the documentation[15] is for an older version, which incorrectly add disjointness within the class hierarchy. This has been corrected for the 3.2.0-rc version used for this comparison.

For stereotypes (item 20), the documentation states that ≪Abstract≫ is allowed on classes and ≪enumeration≫ on datatypes, but that any other usage is discouraged[16]. The tool ignores stereotype when declared as UML stereotype. Regarding ≪enumeration≫, it uses the transformation as described in item 23e.

The built-in dependencies with other files or formats are the configuration files required for the transformation, which are in XML, and the input file is EA XMI only. The configuration files deal with, among others, URIs, version information, imports, and naming the files, as well as processing of datatypes.

The abuse of notation in conventions for UML class diagrams concern the navigability and dependency, which have a different meaning from the UML standard (see items 7 and 8), as per Style Guide [4]. Model2owl is tailored to the Style Guide conventions rather than the UML standard, which is also reflected in the "Constraints/'conventions' on UML diagram and checking for it" feature: they are, and for the SEMIC Style Guide conventions for UML class diagrams specifically. It also generates a compliance report thereof.

The DL fragment could not be determined experimentally. While the generated 'restrictions' file is likely in OWL 2 DL due to the revised processing of multiplicity in 3.2.0-rc (with qualified cardinality, as in item 10), and noting there is also a so-called 'core' (lightweight) version, the generated files in testing (the ePO and a DCAT-AP-LU `.rdf` and `.ttl` files) have, at least, import issues and do not load in the OWL Classifier. Therefore, the DL fragment could not be determined with precision. This may in part be due to rdflib usage (recall page 15 with footnote 2). Meaningfy has a fork of model2owl that uses the OWL API [28], which also has implemented a feature to ignore n-aries, whereas it still reports an error in v3.2.0-rc.

[15] `https://docs.ted.europa.eu/M2O/latest/transformation/transf-rules2.html#_disjoint_classes`; last accessed 4 Feb 2026.

[16] `https://docs.ted.europa.eu/M2O/latest/uml/conv-general.html#sec:stereotypes-tags`; last accessed 4 Feb 2026.

3.2.6 SEMIC/OSLO Toolchain notes

A number of sources were consulted to ascertain the details for the SEMIC Toolchain, observing that some information is more recent than others and may refer to one version or the other[17]. Since the released SEMIC Toolchain (v3 of the OSLO Toolchain) works only with EA v15 files, which the author did not have, the fact-finding for the comparison is based on the documentation and a meeting with its lead developer.

The limited access prevented ascertaining some features for certain. Notably, support for aggregation (items 5 and 6) is unclear, where one source[18] mentions "aggregation", suggesting support in the form of converting it to an object property albeit not how exactly, but not the OSLO-Modelleringsregels, stereotypes for 'modalities' of multiplicities is not mentioned but does appear in practice in, e.g., the DCAT-AP, and likewise for stereotypes on classes, where ≪datatype≫ and ≪enumeration≫ practically are supported. On converting UML associations and association ends (items 2 and 4): the documentation states connector (see footnote 18), not source and target, but elsewhere it does[19], reflecting how the tool has changed practice over the years to follow the SEMIC Style Guide conventions for UML diagrams. We assume the code for this feature has not been removed.

N-aries where n>2 appear not supported, although an association class is possible in the OSLO toolchain documentation (flattened into two binaries). In addition, there are elements supported beyond the SEMIC Style Guide, such as ID and conversion of the 'connector' (association) as well.

UML conventions are described, adhering to SEMIC conventions and additional OSLO toolchain conventions for tag use and checking for valid URIs. By virtue of presumably adhering to the SEMIC conventions, it abuses UML class diagram notation (see items 7 and 8), although it is not documented as such.

The UML formalisation is dependent on whether the UML class diagram is intended as a CV or an AP, and the SEMIC Toolchain lets one choose. For the same `eap` file, if 'vocabulary' is selected for output then it will not add domain and range axioms, according to the developer, although the CVs actually do have them; if 'application profile' is selected, domain and range are declared in the SHACL shape, which they are. It also affects cardinalities. The CVs do not have interesting cardinalities, and its 0..* is converted to a domain and range axiom in a CV, whereas for APs such as DCAT-AP, no Turtle file is generated but rather SHACL shapes and JSON-LD artefacts. An 'exactly 1' is then represented with a `sh:maxCount 1` and a `sh:minCount 1`. The SEMIC toolchain does not officially nor unofficially aim to generate RDF that is compliant with OWL syntax, and therefore N/A was noted. Practically, the

[17] `https://www.vlaanderen.be/digitaal-vlaanderen/onze-diensten-en-platformen/oslo`, `https://github.com/SEMICeu/documentation/blob/main/toolchain.md` and on the same org repo: SEMICeu/uri.semic.eu-generated, SEMICeu/uri.semic.eu-publication?tab=readme-ov-file, and SEMICeu/uri.semic.eu-thema, `https://github.com/Informatievlaanderen/OSLO-EA-to-RDF#readme`, and on the same org repo: /OSLO-UML-Transformer/wiki, /OSLO-UML-Transformer/blob/main/packages/oslo-converter-uml-ea/lib/enums/README.md, and the OSLO-Modelleringsregels available from /OSLO-handleiding/blob/master/Modellering/OSLO-Modelleringsregels.pdf.

[18] `https://github.com/Informatievlaanderen/OSLO-EA-to-RDF/tree/multilingual?tab=readme-ov-file`

[19] `https://github.com/Informatievlaanderen/OSLO-UML-Transformer/blob/main/packages/oslo-converter-uml-ea/lib/enums/README.md`

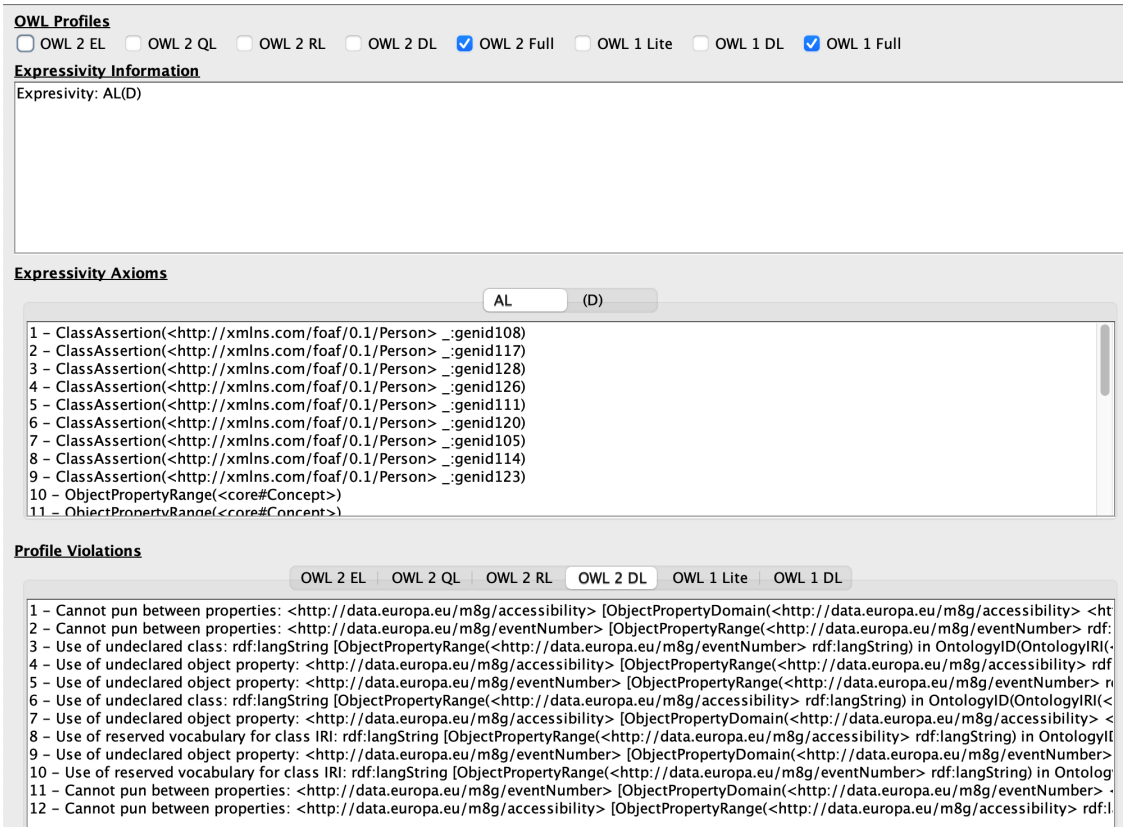

Figure 3.1: The CPEV as assessed in the OWL Classifier: punning violations and misuse of reserved vocabulary push it into OWL 2 Full, and the undeclared elements failed because of missing import declarations where other vocabularies are not the documents they should be. The `genid` entries are due to issues in the declaration of the editors of the file.

CVs are mostly in OWL 2 Full, principally due to syntax issues in the metadata (mainly the author and editor lists) and other infelicities, such as missing imports and use of unsupported datatypes, depending on the CV. The DL features used do not exceed $\mathcal{AL}(D)$. An illustrative example is included in Figure 3.1.

The toolchain uses an internal metamodel in JSON-LD[20], in that first the EA model is converted into JSON-LD and subsequently all artefacts are generated from it. For this reason, the API is a proprietary in-house developed one.

3.2.7 uml2semantics notes

Most features could be determined from the documentation, based on content presented in the reference transformation [27], and later based on running the sample diagram. The lead developer was contacted to get the tool working, and with that revised instruction set[21], it did

[20] `https://github.com/Informatievlaanderen/OSLO-UML-Transformer/wiki/OSLO-JSON-LD`

[21] Specifically: instead of the one on the repo, the following was used: java -jar ./uml2semantics.jar -c "./examples/employer/Employer - Classes.tsv" -a "./examples/employer/Employer - Attributes.tsv" -e "./examples/employer/Employer - Enumerations.tsv" -n "./examples/employer/Employer - EnumerationNamedValues.tsv" -o "./examples/employer/employer.owl" -p "emp:http://uml2semantics.org/examples/employer#" -i "http://uml2semantics.org/examples/employer/v.0.1"

generate the OWL file.

Noteworthy is that it has rules both for association name to object property (item 2 or item 3) and for association end to OP (item 4) with a class expression axiom (item 10), though it was not fully verified in the tool. If only TSV is used, the only option is from association to object property (item 4). Association and attribute reuse (item 15) appears supported in the documentation but is not in the tool when TSV is used; in that case, it ignores any reuse in the behaviour of not adding the axiom to the logical theory.

Regarding UML's multiplicity to qualified cardinality, it maps to unqualified cardinality according to the documentation (see `Thing` usage in [27]), but tool conversion for the provided example shows to qualified cardinality (i.e., item 11 rather than 12).

Overall, the formalisation would be into OWL 2 DL and thus in a fragment of $\mathcal{SROIQ}(D)$. The `employer.rdf` example is in $\mathcal{ALUOQ}(D)$, its provided `dcat-v2.rdf` failed to load in the OWL classifier, and there was limited time to create a diagram and TSV rendering to test the full coverage of supported features. Among others, inverses did not appear to be used, which was not further investigated.

There is no explicit internal metamodel, but it could be argued that the tailored input with the TSV format implicitly acts like one if it were specified more rigorously. It also claims to accept XMI input, but the `-m` or `-xmi` for UML models exported from EA was not recognised and so it did not transform the model.

It has 'other' for stereotypes for types, because it concerns only ≪enumeration≫ on data type, which is converted to one-of (nominals) along the line of item 23b.

On the test whether it generates syntactically correct OWL and the generated file can be processed by other tools: it failed in the (strict) OWL classifier for the `dcat-v2.rdf` sample file, but it loads in the permissive Protégé v5.6.5, albeit with an 'illegal redeclaration of entities' error/warning on `prov:wasRevisionOf` and `prov:specializationOf`, which is not the tool's problem.

3.2.8 LinkML OWL generator notes

LinkML has three options that appear relevant, but only one is for the current scope, which is the OWL generator.[22] Due to time constraints and limited fit of purpose, we only filled in what could be determined from the documentation and the sample files provided, and filling the answers benefitted from the, in part, joint effort with Arthur Schiltz and Emidio Stani who are looking into LinkML as part of a different sub-project within BEACON and zoom in on the RDF generator, and a meeting with the lead developer.

The limited fit requires some caution in reading the table. First, LinkML does not have a graphical editing interface, only image generation from YAML to PlantUML, which has as

[22] Linkml-owl is intended for large ontologies stored in databases to be converted into OWL, which also supports other constructors that the OWL generator does not (see `https://github.com/linkml/linkml-owl`; Last accessed 17 March 2026); e.g., linkml-owl includes the one-of/nominals constructor (the OWL generator does not) and defaults to SomeValuesFrom (the OWL Generator defaults to min 1), and it generates the ontology in functional style syntax (OWL Generator generates Turtle). The RDF generator aims to be an isomorphic translation from YAML into RDF, as a serialisation, to more easily query the models on their properties, such as the number of attributes it has and so on, which may be useful when creating a catalogue of CVs and AP, but not for model conversion as models.

sole purpose to visualise a model and prettify documentation of a model[23]. Therefore, it was decided to not include LinkML in Table 3.1.

Second, for the formalisation choices, we took that to be YAML to OWL and, initially, the online documentation[24]. Due to time constraints, a few questions remain, such as whether there can be 'sub slots' and therewith sub-properties and subsetting for associations in the PlantUML rendering in the YAML file, the treatment of value specifications and whether YAML permits n-aries where n$>$2 (expected value: 'no'). Also, the documentation is limited regarding the conversion rules of the elements, and, moreover, in some cases it is possible to choose how it will be formalised in the list of parameters for the generator, such as min 1 or SomeValuesFrom. It defaults to min 1. In addition, it adds max 1 that is assumed to hold for the YAML slots. Further, while the documentation suggests domain and range axioms are added, this happens only sparingly and if so, property range only. Noteworthy is also the unqualified cardinality to `owl:Thing`, which was common in OWL DL but much less so since OWL 2 DL.

There is no syntax checking also in this tool and so a syntactically correct serialisation is not guaranteed. The sample file provided, as well as an internal test with DCAT-AP-Plus, generated files that were evaluated to be in OWL 2 Full, mainly due to punning issues between properties, undeclared vocabulary use, and occasionally an OWL built-in datatype being used in a datatype definition. Otherwise, the `personinfo.owl.ttl` is in the DL $\mathcal{ALCN}(D)$ and the test file with DCAT-AP-Plus in $\mathcal{ALUN}(D)$. The provided biolink example is beyond OWL with unparsed triples and illegal redeclarations, unsupported data types, and undeclared data properties. Some issues are known and reported on the OWL generator page, notably "For example, if you have slots in your schema that have an Any range, then there is no direct translation of that slot to an OWL-DL property, since each property needs to explicitly commit to being a DatatypeProperty or ObjectProperty in OWL-DL."

Soft imports should be possible through the generator setting to "merge". Lastly, since YAML is the entry and central pivot, LinkML is dependent on it; its metamodel is described in the documentation[25]. The generated output files also include SKOS labels in the annotations, including skos:inScheme, skos:exactMatch, and skos:definition. It also allows for passing on the metadata and to choose to use the OLS metadata from EMBL-EBI for the Ontology Lookup Service catalogue [41].

3.3 Grouping of tools by features

There are a variety of ways to cluster the tools. The following list is a first attempt thereto and may not be exhaustive, and it can be gleaned from the answers in the tables in Section 3.1.

- RDFS (possibly only): INSPEC, Dataspecer, VocBench
- OWL (possibly only): Yhteentoimivuus, model2owl, uml2semantics, VocBench
- SHACL alongside OWL: Yhteentoimivuus, model2owl, LinkML

[23] `https://linkml.io/linkml/generators/plantumlgen.html`; last accessed 17 March 2026.
[24] `https://linkml.io/linkml/generators/owl.html`; last accessed on 2 April 2026.
[25] `https://linkml.io/linkml/schemas/metamodel.html`

- SHACL only, for APs: INSPEC, Dataspecer, SEMIC Toolchain
- Reliance on SKOS: INSPEC, Yhteentoimivuus, model2owl, SEMIC Toolchain, LinkML OWL generator
- Minimal expressivity ($\mathcal{ALH}(D)$ at most): INSPEC, Dataspecer, SEMIC Toolchain
- Expressivity can be minimal, tractable, or high: Yhteentoimivuus, VocBench, model2owl, uml2semantics, LinkML OWL generator
- Syntactically correct output guaranteed through validation: none
- Modelling order enforced or encouraged: INSPEC, Dataspecer, Yhteentoimivuus
- One source from which all other artefacts are generated: model2owl, SEMIC Toolchain, LinkML
- Internal metamodel: Dataspecer, SEMIC Toolchain, LinkML
- Formalisation/generation configurable: SEMIC Toolchain, LinkML OWL generator
- Turtle popularity: Dataspecer, Yhteentoimivuus, VocBench, model2owl, SEMIC Toolchain, LinkML OWL generator
- Domain and range axioms generated by default: Dataspecer, Yhteentoimivuus, model2owl, uml2semantics, LinkML OWL generator
- OP/DP reuse, union of classes: model2owl, SEMIC Toolchain, uml2semantics
- Existential rather than min 1: Yhteentoimivuus, VocBench
- Batch execution: model2owl, SEMIC Toolchain, uml2semantics, LinkML OWL generator
- Open Source: Dataspecer, Yhteentoimivuus, VocBench, model2owl, SEMIC Toolchain, uml2semantics, LinkML OWL generator

4 Discussion

The tool comparison results presented in Chapter 3 revealed substantial differences between the tools, not only regarding features but also underlying philosophy. While the specification and all tools except uml2semantics embed the design decision that there are different types of models and purposes, and therefore have 'variants' in some way, the breadth is wide and none is the same. Two key differences from which several others follow are:

1. *Different models, different formalisations* that may be unconstrained (VocBench) or tightly constrained to what is permissible (Dataspecer, INSPEC) or apply a formalisation pattern at the back-end (Yhteentoimivuus), and in all cases the *choice for a path is made upfront* before creation of the model;

2. *One model, different formalisations for different artefacts* with more or less constraints, and either generated by default (model2owl) or after choosing the type of specification (SEMIC toolchain).

They exhibit **principally different assumptions for working with CVs and APs**, and both have their merits. The Style Guide currently does not include any suggestions on procedures for designing CVs and APs, other than to use *a* methodology—citing **outdated references**—and contains general conventions such as reusing existing concepts as much as possible and styling of a vocabulary definition. One may wish to **consider devising a methodology, procedures, or processes** to assist modellers and developers to design and maintain CVs and APs in the context of SEMIC in particular. For instance, and inspired by the NeOn methodology ontology development [51], to enable choosing one path from a set of "scenarios" to follow, but then with the scenarios tailored to CV and AP development: then there could be, e.g., a path to 'create a CV', one to 'create an AP from a CV', another to 'create an AP from multiple CVs', one to 'use the same modelling language to create multiple artefacts', and so on.

The SEMIC Style Guide aims to cover both CVs and APs, but there is only one list of UML conventions for both, effectively guiding towards the second option listed above. Model2owl and the SEMIC Toolchain operate accordingly, albeit differently; in contrast, Yhteentoimivuus, Dataspecer, and INSPEC enforce the second approach, each in their own way. The two core approaches with variants are visualised in Figure 4.1: INSPEC is of the A1 variety, Dataspecer and Yhteentoimivuus follow the A2 approach to some extent,

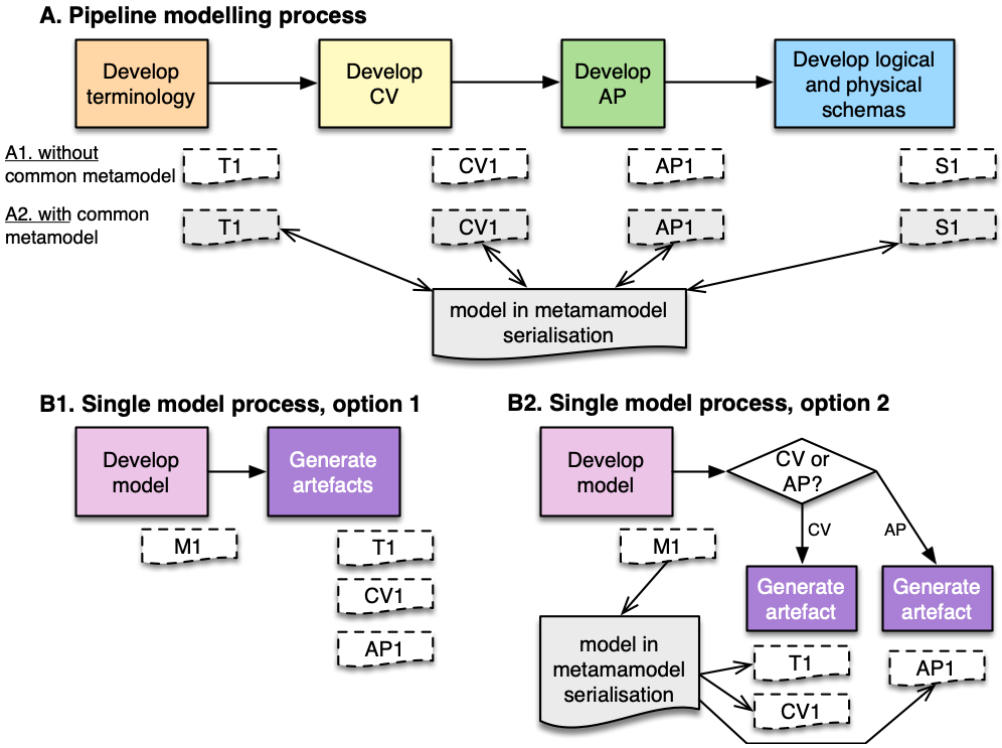

Figure 4.1: The main development processes embedded in the tools: one model after the other in sequence (A) versus one model from which to create all others (B). The terminology and schema development and artefacts (T1 and S1, respectively) are outside the Style Guide's current scope, although ambiguously so for the terminology (possibly as part of the CV).

model2owl uses the B1 variant, and the SEMIC Toolchain and LinkML (roughly) use the B2 variant. This raises the question: **should there be one model in one language from which to generate all artefacts, or different models in different languages?** For the 'one modelling language' option, an obvious parallel to UML Class diagrams is OWL 2 DL, which is sufficiently expressive to cover content for both CVs and APs. For the 'multiple modelling languages' option, the answer is less straightforward and elaborated on further below.

Regarding the 'one model' option, the Style Guide covers the "editorial synchronisation problem" and advocates that "using a pivot representation as a single source of truth is a viable solution". While this is implemented in model2owl, the SEMIC Toolchain, to some extent Yhteentoimivuus[1], and likely holds for LinkML as well[2], this is not the case by design for INSPEC and Dataspecer. Therefore, the question arises: **in accommodating the RDF/RDFS/OWL community, should the "single source of truth" be kept, and if so, what should it be, or should it be abandoned in favour of a pipeline approach, with or without common metamodel?**

Theoretically orthogonal, but practically complicating the answer to the question with the first option is that different languages may, can, and are, being used for different types of formal specifications, and developers of the different tools made different choices that come

[1] it is stored in a central RDF store from which various formats are generated, but one currently cannot go 'back' from an AP in SHACL to changing one's mind and turning it into OWL for a CV.

[2] Equivalence/loss is being looked into by the LinkML team at the time of writing.

with different baggage and decisions baked into the logic. Among others, to formalise APs by design in SHACL only (e.g., SEMIC Toolchain, Dataspecer, INSPEC) and vocabularies in either SKOS, the RDFS fragment of OWL (Dataspecer, INSPEC), or anywhere within OWL 2 (Yhteentoimivuus, model2owl), or adding the notion of terminology in SKOS and then the vocabularies in RDFS (in an OWL file, in case of INSPEC). SKOS and OWL 'live' in the open world; SHACL in a closed world.

How does that matter? Representing, e.g., "Each agent publishes at least one catalogue" in OWL has different logical consequences from representing such information in a SHACL shape, due to reasoning under open versus closed world assumption. In OWL, if either no instances are declared or only `Agent(a1)` is asserted, it will not complain no agent, or `a1`, respectively, is not publishing anything—it could, and that suffices—whereas in SHACL, it will record `a1` as violating the shape because there is no declaration `publishes(a1,c1)` and no `Catalogue(c1)`. In addition, these varied approaches across tools challenge principles of interoperability across tools: a tool that expects only SHACL shapes will not be able to process the knowledge represented in the OWL file (be it as Turtle or another format), and vice versa. The SEMIC Style Guide [3] does not make this clear. At present, everything is assumed to be declared in a SEMIC UML Conventions compliant Enterprise Architect UML Class Diagram, but even then, **it is not specified which UML model content needs to in which type of artefact and in which language** and, consequently, even **the three tools that can process EA UML class diagrams differ in their formalisations and output formats. Likewise, the RDF or OWL-based tools implement distinct decisions that are not compatible**.

The next section elaborates on key differences, issues, and questions these observations raise.

4.1 Issues caused by lack of precision of CVs and APs

The key question arising from the previous section is: **where should what be formalised?** Current practices are:

1. *Two files in the same language*: two OWL files, alike the 'core' and 'restrictions' files that model2owl generates;

2. *One file in one language*: this in possible in Yhteentoimivuus, uml2semantics, and VocBench and holds for the input file of the SEMIC Toolchain and model2owl but not the output;

3. *Multiple files in different languages*: this has three options: i) a lightweight OWL file for the vocabulary *only* and a SHACL shapes file for the AP, as is the case for INSPEC, Dataspecer, and the SEMIC Toolchain, or ii) a lightweight ontology with tractable class expressions and additional SHACL shapes for the AP (Yhteentoimivuus), or iii) a lightweight OWL file for the vocabulary, and generating the restrictions/constraints both in the same OWL language and in SHACL to satisfy different constituencies (model2owl).

The SEMIC Style Guide aims to be for CVs and APs, and thus will need to be more precise on the matter. Whether both CVs and APs easily can fit in a lightweight ontology language (OWL 2 EL, say) depends on the formalisation choices, especially those emanating from property reuse. The union of classes for domain or ranges declaration (item 15b in Section 2.1.1) pushes the formalisation into OWL DL or OWL 2 DL even though very few other language features are used. Such **use also goes beyond what is described in the Style Guide**'s section 8.2, notably regarding the use of domain and range axioms, unions of classes, and simple class expressions (be they of the all-some or all-all pattern).

4.1.1 Transitioning from reuse to individual application

An underlying, more fundamental, question emanating from OWL versus/and/or SHACL is **where to draw the line of an 'open view' and a 'closed view'**, in analogy of the OWA and CWA embedded in reasoning rules and algorithms. The OWA takes that absence of knowledge as unknown (neither true nor false), the CWA evaluates absence to false. The perceived analogy in a model's content is that with an *'open view', the model is extensible and its content applies, or is assumed to hold, across application designs and implementations*, whereas with a *'closed view' (or: single context), that the model's content only applies to one single application and implementation*. For instance, a model about public events that should be applicable to all government departments in all EU Member States—'open' to a number of prospective applications—versus a model that is specific for the design of a database about public events organised by a Department of the Green Economy of Ghent ('closed' to that case only). The intent of openness is evident for CVs, but **APs currently operate in a grey area given the reuse of (part of) APs by other APs: are APs to be reused for multiple applications and implementations?** If so, they would still be 'open', with consequences for the choice of representation language and consequences for how to represent the knowledge. Notably: then SHACL does not seem suitable for representing APs themselves, which conflicts with current practices in INSPEC, Dataspecer, and the SEMIC Toolchain.

The Style guide currently states[3] "An Application Profile is a data specification aimed to facilitate the data exchange in *a well-defined application context*." (emphasis added): text in the remainder of that section suggests it is one design with multiple applications instantiating it, but common practice of APs is reuse, such as can be seen by, e.g., DCAT-AP [44]. Also, some CVs' file names include 'AP', such as the CPEV CV file with name `core-public-event-ap.ttl`, thereby **blurring lines between CVs and APs**. Figure 4.2 visualises the two main differences in interpretation of the Style Guide's content, and consequently tooling that tries to comply with it; e.g., with a four-stage pipeline, one would have:

i) the CV with the entities such as Organisation, organisedBy, and Event, and either domain and range declarations ($\exists$organisedBy.$\top \sqsubseteq$ Event and $\top \sqsubseteq \forall$organisedBy.Organisation) or a class expression with 'each event is organised only by an organisation' (Event $\sqsubseteq \forall$organisedBy.Organisation).

[3] `https://semiceu.github.io/style-guide/1.0.0/terminological-clarifications.html#sec:what-is-an-ap-specification`

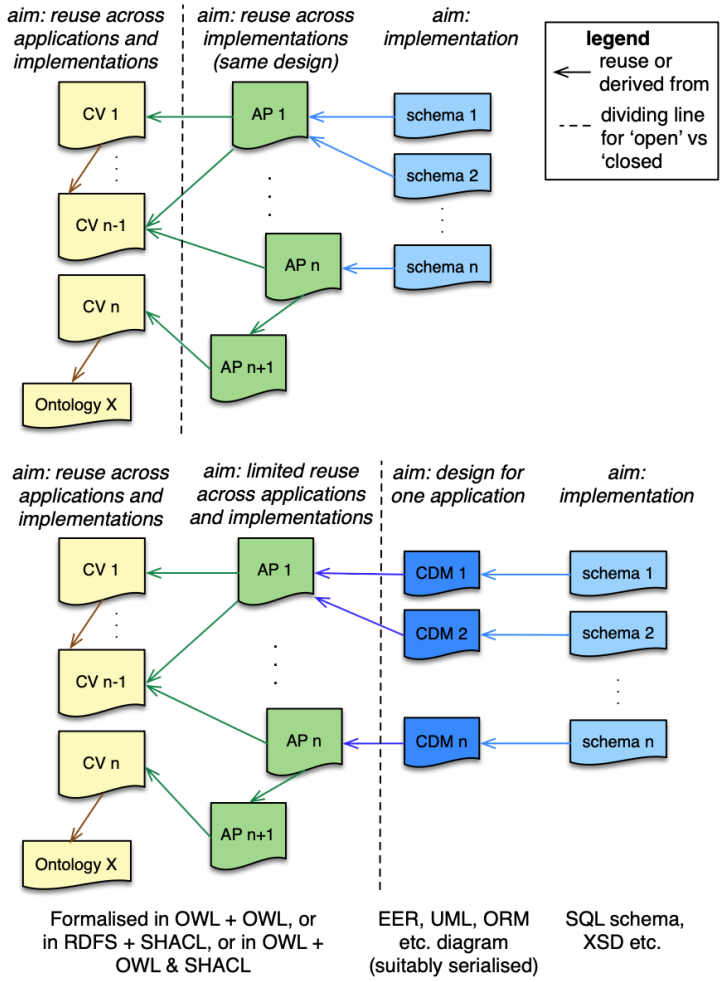

Figure 4.2: Linked artefacts with options where to move to a 'closed' view: for anything on the right-hand side of the dashed line, no constraint may be violated (including no content removed) and absence of information or data means it evaluates to false.

ii) an AP derived from it that also includes a mandatory constraint, such the that 'each event is organised by at least one organisation' (Event $\sqsubseteq \exists$organisedBy.Organisation);

iii) a conceptual data model or data shape specification for some company's software that relies on the AP, whose designers assume they are the only organisers of an event in their prospective database, and so constraining it further to 'each event is organised by exactly one organisation' (Event $\sqsubseteq = 1$ organisedBy.Organisation), and they add an attribute 'each event has exactly one abbreviation STRING' (Event $\sqsubseteq = 1$ hasAbbrv.`xsd:string`);

iv) A physical schema, with an SQL `CREATE TABLE Events` etc., an XSD schema including `<xs:element name="Event" type="EventType"/>`, or a JSON-LD context with a `"@context" { Event : ...}`.

A 3-step pipeline alternative consists of either items i), ii)+iii) combined, and iv), or items i)+ii) combined, iii), and iv). If APs are the key design phase where things are 'closed' and only

implementations are allowed to be derived from it, then that company would not be allowed to do what it did in the third stage, above. Consider, for instance, a WordPress installation: someone somewhere designed the system, and many people install it out-of-the-box, and the model for the database back-end is the same throughout. Is an AP a blueprint like that? If so, then one's assumption about APs is along the line of the top-half of Figure 4.2.

Consider now you are developing an XXX-AP-yourCountryAbbreviation: constraints such as in iii) could be included in the AP or are required because of national legislation (e.g., DCAT-AP-LU has Dataset has publisher 1..1 Agent, whereas DCAT-AP has 0..1), suggesting the line is as in the bottom-half of Figure 4.2, like that event organisation company above, with consequences to favour a more permissible representation language to allow for changes, i.e., OWL, which is possible in Yhteentoimivuus, VocBench, model2owl, uml2semantics, and LinkML's OWL generator.

Either way, overall, it can be argued that **the position of that dotted line in Figure 4.2 governs the freedom, or lack thereof, to modify and adapt the artefacts for one's specific context, application, and implementation, and thereby the representation language**.

Finally, given AP reuse, even if it is limited reuse, there are **questions about how to reuse it and how to resolve conflicts if the new AP inherits from two APs with conflicting content**, for which there is no Style Guide guidance on what to do. For instance, consider again our XXX-AP-yourCountryAbbreviation that now has to be compliant with both DCAT-AP and with HealthDCAT, or a profile about machine learning data for health, reusing the Health and ML DCAT specifications. Aside from the fact that there is currently no infrastructure to check compliance automatically, it is unclear how to resolve conflicting constraints or content, like, say, a precedence ordering of APs or else an 'up to the modeller' guideline.

4.1.2 How to formalise it also depends on closed vs open view

A second issue of the where to 'close' it, or to permit only a single context, concerns domain & range declarations versus class expressions (items 9 and 12 versus 10 and 11 in Section 2.1.1), the notion of reuse of object and data properties (item 15 in Section 2.1.1), and commitment to datatypes, as was also illustrated in Figure 2.3 in Section 2.1.1. The former influences the formalisation option, and language. First, to recap:

- *RDFS is intended for information modelling* for one application, i.e., in a 'closed' setting, and therefore one can, and is encouraged to, declare domain and range axioms, because extensibility and the 'outside' do not matter (anymore);
- *OWL is intended for representing knowledge* in a open and extensible way (and also can be used for modelling information for a single application). In the extensible way, one uses class expressions, so that object and data properties can be reused across contexts and applications.
- OWL *can be repurposed* to a large extent for the closed view, by using the domain and range axioms for the object properties and data properties, and data type declarations. It also requires one to manually declare classes outside a class hierarchy to be mutually disjoint (item 19), carrying over the assumption from UML class diagrams.

- RDFS *might be argued to be repurposable* for reusable information or knowledge through a workaround by using the `domainIncludes` and `rangeIncludes` annotation properties with no logical consequences, following on from Schema.org that uses it as a property on a property to relate it to a class.

Among the formalisation practices (see Table 3.3), there are no evident clusters where more than one tool consistently takes the same approach.

For the data types:

- Commitment to a data type is *necessary* for the physical schema. It is also required in UML class diagrams and for ORM's value types.
- Commitment to a data type is *obstructive* in a CV and possibly also an AP, because one does not know how the data will be stored in a particular application[4]: e.g., a database may use a `STRING` as data type for some attribute, which clashes with a requirement for `xsd:string` dictated by an AP. Must it clash and subsequently comply in the company's database, rather than letting them choose to create an API and include a conversion step, leaving their database intact and as they want to have it internally within the organisation?

In addition, these issues become challenging in the details in the formalisation. For instance, LinkML's YAML models unapologetically reside in the closed view, yet embedded that in part in the formalisation step: it assumes any property has a 'max 1' and this is expressed so in the generator rules. This is the assumed default for attributes in conceptual data models such as UML, but not for relationships/associations and it is not commonly declared for all object and data properties in ontologies or vocabularies, nor is this the default case for CVs and APs. Yet, the OWL generator does not follow through with the 'closed view' on the assumed class disjointness, nor a union of classes for domains or ranges.

Data shapes generation and validation

Additional consequences relate to the "data shapes": if the shapes refer to SHACL shapes, observe that it is, by design and assumption, a *post hoc* validation mechanism. While one may want to generate shapes for APs to assist checking graphs, if all constraints of APs are in data shapes only, this complicates implementation of constraints in non-RDF data stores and application software, where integrity constraints control data entry and manipulation to prevent violations[5] and comply with the constraints upfront and thus need a re-implementation of the data shapes. Further, **SHACL generators and validators differ** on whether they support fragments of SHACL, SHACL core specifically, or unrestricted SHACL (i.e., with recursion and negation [8]), core or SHACL-SPARQL, or the draft v1.2, and with or without reasoning [31], and with or without adhering strictly to the list of UML constraints supported in the Style Guide.

[4] For a longer explanation, see the "attributions" subsection in Section 6.1.1 of [32].

[5] For example, checking the data type upfront upon entry (e.g., disallowing inserting a tuple with a cell value that is a string when integer is specified in the SQL schema) and referential integrity with the foreign key mechanism in relational database management systems.

Let us illustrate two issues, one for shape generation and one for validation. DCAT-AP v3.0.1 contains a data type TemporalLiteral that is described as "rdfs:Literal encoded using the relevant ISO8601 Date and Time compliant string and typed using the appropriate XML Schema datatype (xsd:gYear, xsd:gYearMonth, xsd:date, or xsd:dateTime)." [44]. The corresponding SHACL shape file[6] does *not* contain `TemporalLiteral`. Instead, where it is used in a class, such as for `Catalog_Shape`'s `modified` property, it has

```
sh:maxCount 1 ;
       sh:node :DateOrDateTimeDataType_Shape ;
       sh:path dct:modified ;
```

where that node `DateOrDateTimeDataType_Shape` is the union of the four data types, being (label and comments removed for brevity):

```
:DateOrDateTimeDataType_Shape
   a sh:NodeShape ;
   sh:or ([ sh:datatype xsd:date ]
          [ sh:datatype xsd:dateTime ]
          [ sh:datatype xsd:gYear ]
          [ sh:datatype xsd:gYearMonth ] ) .
```

The SEMIC Style Guide does *not* include the union operator as a supported language feature and another SHACL shape generator tool that consequently does not implement `sh:or` will resort to a workaround, such as selecting only one of the four data types or implementing it via a class or sub-datatypes of leaving it blank; therewith, it generates a different shape and sets different expectations on the data shapes to validate. *Even a basic SHACL validator will then return different pass/fail outputs on the same data graph*.

Regarding the reasoning, first, note that foaf:Organization is a subclass of foaf:Agent, and an AP includes foaf:Agent with a few constraints, such as in DCAT-AP:

```
:Agent_Shape
    a sh:NodeShape ;
    rdfs:label "Agent"@en ;
    sh:property [
        sh:minCount 1 ;
        sh:nodeKind sh:Literal ;
        sh:path foaf:name ;
        sh:severity sh:Violation
    ], [
        sh:maxCount 1 ;
        sh:path dct:type ;
        sh:severity sh:Violation
    ] ;
    sh:targetClass foaf:Agent .
```

[6] `https://semiceu.github.io/DCAT-AP/releases/3.0.1/html/shacl/shapes.ttl`

If a graph contains data about the EU as an instance of foaf:Organization in the position where the above `:Agent_Shape` applies, a 'simple' SHACL validator will output a *violation* because the EU is *not asserted* to be an instance of foaf:Agent. In contrast, in the presence of the vocabulary and basic automated reasoning services, the data graph will *pass* as satisfying the shape, because in validating, it *infers* that the EU is an instance of foaf:Agent thanks to the rules of subsumption. Consequently, **the same UML class diagram may result in different shapes and the same graph may pass one validator but not another when it is checked against the 'same' AP**.

The current comparison did not include an assessment on whether SHACL shape generation differs across the tools. **If SHACL is to be promoted as only formalisation for APs, these issues must be resolved to foster compatibility, interoperability, and trust in the systems**. If there is a common source specification and the AP is a model on its own, it is still highly advisable to provide guidance on SHACL shape generation.

4.2 Modelling conventions and metadata

Modelling conventions, if described, are hardly ever fully fine-grained, at least not at the level of detail as the SEMIC Conventions for UML class diagrams. INSPEC has a few clear instructions (e.g., one URI/IRI per vocabulary), but also lacks fine-grained aspects such as whether each vocabulary element must have a definition, naming conventions (camel case, spaces in names) etc. There are arguments in favour and against doing so; either way, one has to **decide on how detailed the prospective conventions for the semantic web language(s) should be**. Currently, the Style Guide has conflicting information, from an informal and arguable alignment in its section 7.2, concrete elements but not how they are to be used in its section 8.2, and a non-aligned list in its terminology section for OWL. There are no such details at any level of detail for the SHACL shapes.

Related to the modelling conventions are ontology design patterns. While no tool has official design patterns, practically they can be enforced through modelling conventions or restrictive GUIs, which both Yhteentoimivuus and Dataspecer avail of. This can be tied to the need to **specify good modelling patterns and contrast them with anti-patterns to avoid**. For instance, to formalise a 1..* in a visual model with an "$\exists$" (a `SomeValuesFrom` in the serialisation) instead of a ≥ 1 (an `ObjectMinCardinality`) in a CV or AP.

While a majority of tools have 'yes' for the ability to declare metadata, and the SEMIC Toolchain also a 'yes' for the Semantic Registry Model (SRM), based on cursory observation, the **amount of metadata that is or can be declared is often limited**, thereby having the consequent effect that it will be harder to find and query CVs and APs. Also, it was a recurring question in the conversations with the tool developers what SRM was.

It may be considered within the SEMIC Style Guide scope to review and determine a minimum set of metadata features for both the RDF and OWL-oriented tools and for the UML-to-OWL and UML-to-SHACL tools, not only for the "human readable form" (Section 10.6 of the Style Guide). Doing so also requires specification of the **purpose of the metadata**: who or what is it intended for? What is expected to be possible with it, what is the minimum metadata that should be recorded? And how will the existing artefacts be made compliant with that? For instance, if CVs and APs are considered 'data', each such artefact

could be annotated with DCAT-AP; if the CV or AP is seen more like a lightweight ontology, suggestions such as (a subset of) the MIRO guidelines [40] may be of use; if the metadata is to serve managing a repository, then any of the many conceptual data models or database schemas for a repository (BioPortal, OLS, OntoHub, ROMULUS, etc.) may be of use in relation to SRM. Currently, only the SEMIC Toolchain and LinkML's OWL generator have a systematic and dedicated feature for it (SRM and OLS's metadata requirements, respectively). The SEMIC **CVs and APs may benefit from a minimal agreed-upon list or model for metadata generation and promote this more prominently in the Style Guide**, especially for a prospective SEMIC artefacts catalogue, and to ensure that the metadata generation does not modify the language that the CV or AP is represented in[7].

4.3 Internal metamodel as pivot and its expressiveness

The internal metamodel question, i.e., on whether transformations are governed by a central model as pivot, was set out to be only one of yes/no, and it turns out only a few tools have one, notably LinkML, the SEMIC Toolchain, and Dataspecer. Given that an internal metamodel as pivot is technologically easier to maintain and allows more easily for extensions, it may be beneficial to investigate this further, and, firstly, to look into the **comprehensiveness of the metamodels** across the tools that have one, because it governs what can be done with a tool overall, including what language features can be used. For instance, if the metamodel only permits domain and range axioms but not class expressions, then no serialisation will permit class expressions with cardinality constraint no matter the expressivity of the target language. This may also involve accommodations for the various tags that can be useful to manage generating documentation and supporting multilingual specifications. In addition, depending on the design, it is also possible to govern the amount of metadata through the metamodel.

If this will be considered, it would be useful to include the format and documentation of the metamodel. LinkML and the SEMIC Toolchain maintain it on relatively dynamic GitHub pages, whereas Dataspecer's DSV also has an archival version[8]. Besides the minimum content, another question arises: **should it be a metamodel for SEMIC's CV, AP, and derivative artefacts** to facilitate the generation of a number of artefacts and be robust to the whims in IT of changing serialisation formats, **or one for UML and the RDF or OWL plus mappings into UML and RDF or OWL only?**

If the metamodel-based approach is too daunting or beyond the available resources, then a tool-independent UML serialisation may be an avenue to explore, likely as **XMI without the XMI extension mechanism or with SEMIC-specific extensions**. Such an XMI specification then permits use of any UML modelling tool and developers need only to design a set or rules to-from that prospective XMI specification.

[7] that is, if the artefact is represented in SHACL, then it should be syntactically correct SHACL, not claiming SHACL is no more than RDF and the output is an RDF graph and that RDF is valid; see also Section 4.4.

[8] Klímek, J., Stenchlák, Š., Škoda, P. (2026). Data Specification Vocabulary (DSV): Representation of Application Profiles of Semantic Data Specifications. In: Pardede, E. et al. (eds). Information Integration and Web Intelligence (iiWAS'25). LNCS vol 16330, pp221-236. Springer. DOI: 10.1007/978-3-032-11976-6_16. (paywalled)

4.4 Observations of a practical nature

Most tools generated different output formats, and notably in Turtle and RDF syntax, and several tools also can generate JSON-LD output (not further reported on). **Most tools do violate syntax** in one way or another, however, and for different reasons. **No evaluated tool carries out any syntax checking**. Related to this issue, is that some **tools produce files with problematic IRIs**, including pointing to non-existent locations by default, inserting IRIs to unused or deprecated documents, circular imports, and different vocabularies (in different documents) pointing to the same IRI. This needs to be resolved and will affect both the tooling infrastructure as well as the current use of the CVs and APs.

Tools that were not expected to have a visual interface and be evaluated on it, were after all (Dataspecer, Yhteentoimivuus, LinkML, EntryScape), using the features as a UML-equivalent, suggesting that at least domain experts, if not also modellers themselves, prefer **visualisation of the model** as a mode of interaction. This suggests that even when the Style Guide is extended for RDF, RDFS, SHACL and/or OWL, guidance on visual and/or textual rendering will be appreciated by the community. It also entails that there is an obfuscation step that makes it more difficult to assess the exact formalisation taking place.

While the respective developers contacted were very responsive, the online documentation does not necessarily make easily clear who is responsible or should be contacted. For instance, model2owl's readme page states at the very bottom of the long page[9] "When contributing to this repository, please first discuss the change you wish to make via issue, email, or any other method with the owners of this repository before making a change.", but no names or emails are given there, and for the SEMIC Toolchain one is pointed to an issue tracker on GitHub only[10], noting that to add an issue one has to have an account and be logged in. Conversely, INSPEC[11] does list both a link to a GitHub repo and an email address. Related to this software infrastructure aspect, is the widespread use of GitHub, with all that comes with it. In the light of the drive for open source and for sovereignty, it is advisable to **reconsider GitHub as core infrastructure and whether it serves its purpose as repository of CVs and APs**. For instance, one could develop a repository dedicated to SEMIC only or also SEMIC-aligned CVs and AP, not unlike BioPortal [52] for biomedical ontologies or the Ontology Lookup Service OLS4 of the European Bioinformatics Institute [41].

Lastly, and perhaps ironically, tooling and documentation of the tools that were compared use different terminology where the same artefacts or elements are meant. First, for the Style Guide to become independent of EA yet still accepting UML models, **the Style Guide requires corrections to the UML terminology to adhere to the UML Specification**'s terminology, such as using "association" and "association end" rather than "connector", "source", and "target". This varies by tool documentation at present, where uml2semantics adheres to UML terminology and others to the EA terminology (model2owl, notably). Second, several types of artefacts are described in the Style Guide, and **it is not clear whether SEMIC compliant tools would need to use the same terminology for the artefacts**. For instance, one tool could use maybe "data vocabulary" or "ontology" for SEMIC's "core vo-

[9] `https://github.com/OP-TED/model2owl/`; last accessed on 7 April 2026.

[10] `https://semiceu.github.io/toolchain-manual/`; last accessed on 7 April 2026.

[11] `https://diggsweden.github.io/interoperable-specifications/`; last accessed on 7 April 2026.

cabulary" in the interface, or "data model" for SEMIC's "application profile". In the general case, this could be left to the tool developer; alternatively, the Style Guide could make a recommendation and/or **extend the list of synonyms of the various artefacts listed in its "Terminological clarifications" section**.

5 Closing remarks

The comparison of seven tools and a specification showed a wide variety of design and implementation decisions, all trying to fill perceived gaps where the SEMIC Style Guide did not guide. Even the two tools explicitly stating to adhere to the SEMIC Style Guide conventions with UML differ substantially in design principles and artefacts that the tools generate.

Main directions for follow-up tasks to update and extend the Style Guide are to better demarcate what language features may be used in a CV and what in an AP and where the boundary between reuse and once-off application context begins, from which the key language decisions follow and, from there, respective syntax checks. Other tasks concern the choice for one metamodel, minimal metadata guidelines, and possibly a dedicated repository with CVs and APs. Finally, there are several tasks for editorial improvements, including, among others, addressing the GitHub issues, correcting some errors, adding explanatory content to examples and the reuse section, and updates to the references.

Acknowledgements

We would like to thank the workgroup for input: Anastasia Sofou, Sander van Dooren, Jitse de Cock, Bert van Nuffellen, Emidio Stani, and Maria Keet. Our gratitude goes out to the tool developers contacted, for their extensive feedback in writing and/or demonstrating the tool, in alphabetical order: Riitta Alkula (Yhteentoimivuus), Henriette Harmse (uml2semantics), Jakub Klímek (Dataspecer), Grzegorz Kostkowski (model2owl), Chris Mungall (LinkML), Bert van Nuffellen (SEMIC/OSLO Toolchain), and Matthias Palmér (INSPEC and EntryScape). We also thank the input by Arthur Schiltz and Emidio Stani for input to the LinkML content and Jana Ahmad for contributions to model2owl, dataspecer, and VocBench, and Csongor Nyulas and Eugeniu Costetchi for reviewing an earlier draft.

A

Feature lists of RDFS and OWL

Visually, in short, the languages relate as shown in Figure A.1, with the caveat that it is very easy to make syntax mistakes with RDF and RDFS, and therefore the 'RDFS only' easily ends up in OWL Full or OWL 2 Full. The lists of features were copied from Keet's ontology engineering course slides[1] and adjusted for formatting to suit the layout of this appendix.

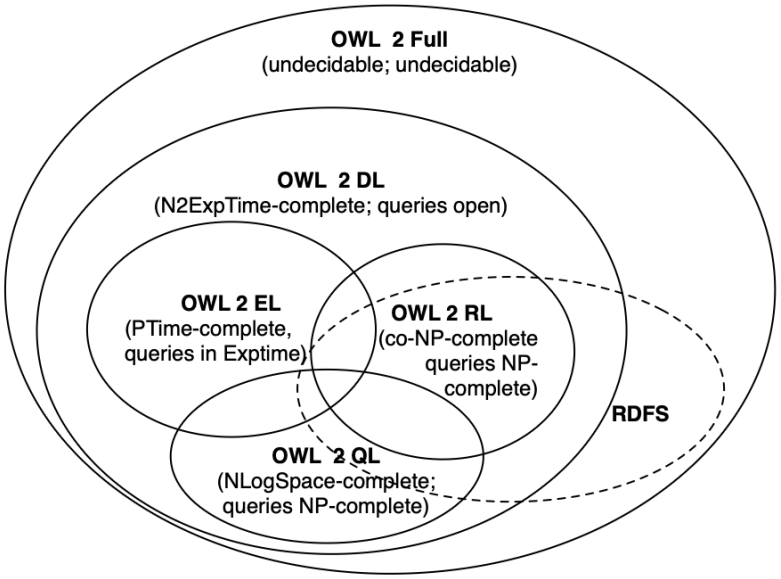

Figure A.1: Venn diagram of the OWL 2 Species and RDFS.

RDF Schema

RDFS is aimed at being a "data-modelling vocabulary for RDF data" and is a semantic extension of RDF. RDFS has the following key feature for declaring information:

- Classes

[1] OWL latex source files: `https://people.cs.uct.ac.za/~mkeet/OEbook/`; last accessed 4 May 2026.

- Properties
- Class hierarchies
- Property hierarchies
- Domain and range restrictions
- Non-normative aspects, such as RDF containers (resources that are used to represent collections, such as bag and set) and a reification vocabulary.

Conversely, there are *limitations to RDFS*, which may serve to point out:

- It has local range restrictions (e.g., for a class Person, the property hasName has range `xsd:string`), which restricts reuse of the property.
- On cannot declare complex concept descriptions (e.g., to have Father be defined by being the intersection of Parent and Male)
- It lacks cardinality restrictions (e.g., that each Company is headquartered in exactly 1 Location)
- One cannot declare disjointness axioms (e.g., that no Company can be both a Non Profit Company and a For Profit Company)
- It permits only binary relations
- There are no characteristics of properties (e.g., inverse, transitive)

In addition, there are layering issues regarding syntax and semantics. Regarding the RDF syntax, RDF allows only binary relations, the syntax is verbose, there are no limitations on the graph in RDF (i.e., every graph is valid no matter how non-sensical). Regarding semantics, interpreting malformed graphs is problematic, one can use vocabulary in the language when one should not (e.g., `rdfs:Class rdfs:subClassOf ex:a`), and, generally also unintentionally, declare meta-classes in the wrong way (e.g., `ex:a rdf:type ex:a`) and still be an RDF graph.

OWL 2

OWL 2 consists of OWL 2 DL, and three profiles, being OWL 2 EL, OW 2 QL, and OWL 2 RL, as well as an OWL 2 Full that permits the same problematic language use as described in the RDFS section, above.

Since lightweight ontologies are within the scope of SEMIC, we fist list the profiles' respective features, and then those of OWL 2 DL

OWL 2 profiles

The profiles are listed in order of expected use for the SEMIC context.

OWL 2 EL

Supported class restrictions on OWL 2 EL (in short: $\sqcap\ \exists\ \top\ \bot \sqsubseteq \sqcap\ \exists\ \top\ \bot$):

- existential quantification to a class expression or a data range

- existential quantification to an individual or a literal
- self-restriction
- enumerations involving a single individual or a single literal
- intersection of classes and data ranges

Supported axioms, restricted to allowed set of class expressions:

- class inclusion, equivalence, disjointness
- object property inclusion and data property inclusion
- property equivalence
- transitive object properties
- reflexive object properties
- domain and range restrictions
- assertions
- functional data properties
- keys

A number of OWL 2 DL features are NOT supported in OWL 2 EL, being: universal quantification to a class expression or a data range, cardinality restrictions, disjunction (union), class negation, enumerations involving more than one individual, disjoint properties, irreflexive, symmetric, and asymmetric object properties, and inverse object properties, functional and inverse-functional object properties.

OWL 2 QL

Supported Axioms in OWL 2 QL, restrictions:

- Subclass expressions restrictions:
  - a class
  - existential quantification (ObjectSomeValuesFrom) where the class is limited to owl:Thing
  - existential quantification to a data range (DataSomeValuesFrom)
- Super expressions restrictions:
  - a class
  - intersection (ObjectIntersectionOf)
  - negation (ObjectComplementOf)
  - existential quantification to a class (ObjectSomeValuesFrom)

  - existential quantification to a data range (DataSomeValuesFrom)

Supported axioms in OWL 2 QL:

- Restrictions on class expressions, object and data properties occurring in functionality assertions cannot be specialized
- subclass axioms
- class expression equivalence (involving subClassExpression), disjointness
- inverse object properties
- property inclusion (not involving property chains and SubDataPropertyOf)
- property equivalence
- property domain and range
- disjoint properties
- symmetric, reflexive, irreflexive, asymmetric properties
- assertions other than individual equality assertions and negative property assertions (DifferentIndividuals, ClassAssertion, ObjectPropertyAssertion, and DataPropertyAssertion)

NOT supported in OWL 2 QL are: existential quantification to a class expression or a data range in the subclass position, self-restriction, existential quantification to an individual or a literal, enumeration of individuals and literals, universal quantification to a class expression or a data range, cardinality restrictions, disjunction, property inclusions involving property chains, functional and inverse-functional properties, transitive properties, keys, and individual equality assertions and negative property assertions.

OWL 2 RL

Supported in OWL 2 RL are:

- OWL 2 DL but with long list of restrictions on class expressions, which are included in Table 2 at `https://www.w3.org/TR/owl2-profiles/#OWL_2_RL` (e.g., no SomeValuesFrom on the right-hand side of a subclass axiom). All axioms in OWL 2 RL are constrained in a way that is compliant with the restrictions in that Table 2.
- In short: OWL 2 RL supports all axioms of OWL 2 apart from disjoint unions of classes and reflexive object property axioms, and no $\forall$ and $\neg$ on the left-hand side of the inclusion, and $\exists$ and $\sqcup$ on the right-hand side of $\sqsubseteq$.

OWL 2 DL

OWL 2 DL has the following modelling features:

- All features of OWL DL (see below)
- Qualified cardinality constraints
- More property characteristics: reflexive (local and global), irreflexive, asymmetric
- Property chains
- A `Haskey` 'key' (note: it doesn't behave like databases keys)
- Richer datatypes, data ranges
- Fancier metamodelling and annotations and improved ontology publishing, imports and versioning control

OWL

For the sake of completeness, OWL Lite, OWL DL, and OWL Full are included here, but one is discouraged to use either of them in the 2004 serialisation.

- **OWL Lite**
  - (sub)classes, individuals
  - (sub)properties, domain, range
  - conjunction
  - (in)equality
  - (unqualified) cardinality 0/1
  - datatypes
  - inverse, transitive, symmetric properties
  - someValuesFrom
  - allValuesFrom
- **OWL DL**
  - All of OWL Lite
  - Negation
  - Disjunction
  - (unqualified) Full cardinality
  - Enumerated classes
  - hasValue
- **OWL Full**
  - Meta-classes
  - Modify language

B

A07.02: Report on the benchmarking of tools: outcome table

The tool-based evaluation in the “A07.02: Report on the benchmarking of tools” focussed on high-level tool features as listed in Table B.1 and the outcome table is included as a screenshot in Table B.2. Refer to the report for details.

Table B.1: Feature list for the benchmarking task.

Functionality	**Requirement**	**Method of evaluation**
Generate HTML	Custom	Yes/No
	ReSpec style	Yes/No
	Integrated examples	Yes/No
	Multilingual HTML	Yes/No
	Integrated validation	Yes/No
Generate Machine readable artefacts	RDF	Yes/No
	SHACL Shapes	Yes/No
	JSON-LD Context	Yes/No
	XSD	Yes/No
Ingest	Type of inputs	List
	Enterprise Architect	Yes/No
	OS UML Editor	Yes/No
Metamodel	Type of metamodel	Explicit/Implicit*
	Documentation	Poor/Good/Strong**
Tools	Type of interface	GUI/CLI/IDE
	Dependencies	List

Table B.2: Results of the benchmarking task.

Functionality	**Requirement**	**LinkML**	**BESSER**	**Dataspecer**	**model2owl**	**SEMIC Toolchain**
Generate HTML	Custom	Yes	No	No	Yes	Yes
	ReSpec style	Yes*	No	Yes	No	Yes
	Integrated examples	Yes	No	No	No	Yes
	Multilingual HTML	Yes	No	No	No	No
Generate Machine readable artefacts	RDF	Yes	No	Yes	Yes	Yes
	SHACL Shapes	Yes	No	Yes	Yes	Yes
	JSON-LD Context	Yes	No	No	No	Yes
	XSD	No	No	Yes	No	No
Ingest	Type of inputs	YAML	JSON, B-UML	No	UML (EAP)	UML (EAP)
	Enterprise Architect	No	No	No	Yes	Yes
	OS UML Editor	No	No	No	No	No
Metamodel	Type of metamodel	Explicit	Explicit	Implicit	Explicit	Implicit
	Documentation	Strong	Good	Poor	Good	Good
Tools	Type of interface	IDE, CLI	GUI, IDE	GUI	GUI, IDE	GUI, IDE
	Dependencies	Python, MkDocs, Mermaid	Docker, Python	Docker	GiHub	CircleCI, Docker

C

Harvested tools not included in the comparison

The following UML-to-OWL tools were found in the time available, but not included mainly because they were either old and supporting only older XMI versions, not maintained, inaccessible by now, or did not satisfy the open source requirements:

- UML2OWL - Gasevic. https://www.sfu.ca/~dgasevic/projects/UMLtoOWL/; input: UML/XMI with Ontology UML Profile (OUP); output: OWL ontology; notes: XSLT-based UML to OWL converter, tested only with older UML tools (e.g. MagicDraw, Poseidon), research prototype, not actively maintained.
- CIM tool. https://cimtool.ucaiug.io/further-learning/uml-xmi-owl/; input: UML XMI; output: OWL ontology (RDF/OWL); notes: paper at https://link.springer.com/chapter/10.1007/978-3-642-10583-8_16.
- Cameo concept modeler. https://docs.nomagic.com/spaces/CCMP2021x/pages/64972427/Cameo+Concept+Modeler+Documentation; input: UML models (MagicDraw/Cameo); output: OWL 2 ontologies (RDF/XML, Turtle, JSON-LD, OWL Functional, Manchester); plugin for MagicDraw; notes: UML-based concept modelling plugin; supports UML to OWL and back round-tripping, glossary generation, and semantic annotations; not based on open-source UML tools.
- UML2OWL - Grunwald. https://sourceforge.net/projects/uml2owl/ (old location: 404 for http://www.umlowlgen.com/); input: UML XMI (Visual Paradigm XMI 2.1 – and MS Visio XMI 1.0 and ArgoUML XMI 2.1 as well?); output: OWL ontology; notes: Java-based UML to OWL converter, supports multiple XMI versions, focuses on reliability of specific UML tool exports, legacy tool (last update 2013), and a technical report is available at https://qse.ifs.tuwien.ac.at/publication/IFS-CDL-14-42.pdf.
- UML2OWL - Xu et al. no tool availability; input: XMI 2.0; output: OWL DL ontology; notes: Xu, Z., Ni, Y., Lin, L., and Gu, H. A Semantics-Preserving Approach for Extracting OWL Ontologies from UML Class Diagrams. Database Theory and Application (2009), 122-136.

- UML2OWL - Leinhos. http://diplom.ooyoo.de/index.php?page=about; input: XMI 1.2 as implemented by Poseidon CE 4.1; output: OWL (OWL DL, RDF/XML); notes: Leinhos, S. OWL Ontologieextraktion und -modellierung auf der Basis von UML Klassendiagrammen. Master's thesis, University of the Federal Armed Forces Munich, 2006.

- Eclipse ATL. https://eclipse.dev/atl; input: UML 2.0 models (XMI / EMF); output: Models conforming to a target metamodel (e.g. OWL metamodel, Ecore, XMI); notes: General-purpose M2M transformation language, UML to OWL supported with ODM use case (https://eclipse.dev/atl/usecases/odmimplementation/).

- UML2OWL - Solores. https://w3.ual.es/acg/soleres/ieee-smca/; input: XMI; output: OWL; Eclipse plugin; notes: Open-Environmental Ontology Modeling. Luis Iribarne, Nicolas Padilla, José Andrés Asensio, Javier Criado, Rosa Ayala, Jesus Almendros, and Massimo Menenti. IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS–PART A: SYSTEMS AND HUMANS, VOL. 41, NO. 4, JULY 2011.

- ODM Implementation (Bridging UML and OWL). https://eclipse.dev/atl/usecases/odmimplementation/; input: XMI 2.1; output: OWL model (OWL metamodel) to OWL/XML; Eclipse UML2 plugin or Papyrus; notes: Implements OMG ODM mappings using ATL, supports UML classes, attributes, associations, and instances, bridges MDA tools and Semantic Web tools

- Dia UML2OWL. https://projects.gnome.org/dia/; input: UML diagrams (Dia internal XML; MS Visio via VDX import); output: OWL (RDF/XML); notes: XSLT-based converter bundled with Dia.

- CODIP. https://projects.semwebcentral.org/projects/codip/ (does not work); input: UML via XMI 1.2; output: OWL ontology; notes: Java-based transformation using NetBeans MDR, outdated, limited XMI support.

- EulerGUI. https://eulergui.sourceforge.net/; input: UML XMI 2.1; output: N3 / OWL (via conversion); notes: Rule-based reasoning environment, UML to OWL transformation indirect.

- OntoStudio. https://semafora-systems.com/en/products/ontostudio/ (404 error); input: UML 2.0 (XMI); output: ObjectLogic (not OWL); notes: Commercial semantic modelling environment, no UML to OWL support.

- ICom. https://www.inf.unibz.it/~franconi/icom/; input: XML (native serialisation); output: Ontologies with DL reasoning support (ALCQI); stand-alone; notes: UML directly into OWL during the modelling activity, paper available at https://journals.sagepub.com/doi/10.3233/SW-2011-0038

- crowd2. https://crowd-app.fi.uncoma.edu.ar/; input: JSON-serialised UML, ER, ORM (own syntax); output: JSON (notation-specific), OWL (incl. OWL/XML), reasoning support; web-based; notes: Multi-modal modelling tool with reasoning support,

focuses on interactive modelling and transformation between notations, not on importing UML/XMI files, UML to OWL converter in modelling environment, relevant for conceptual-level comparison, and UML directly into OWL during the modelling activity

Reference formalisations and comparisons to note:

- Daniela Berardi, Diego Calvanese, and Giuseppe De Giacomo. Reasoning on UML class diagrams. Artificial Intelligence, 168(1-2):70-118, 2005.
- Andreas Grünwald. Evaluation of UML to OWL Approaches and Implementation of a Transformation Tool for Visual Paradigm and MS Visio. Vienna University of Technology, Austria. Technical Report No. IFS-CDL 14-42. July 2014. `https://qse.ifs.tuwien.ac.at/publication/IFS-CDL-14-42.pdf`

Possibly relevant but neither UML nor OWL as input:

- LinkML. `https://linkml.io/`; input: YAML; output: RDF, JSON-LD Contexts, JSON-LD, SPARQL, ShEx, SHACL, and OWL; notes: Data modelling language independent of UML, supports linked-data concepts, can generate OWL and RDF artefacts but does not consume UML.
- ORMie. `https://github.com/ormsolutions/NORMA`; input: Object-Role Modelling models, with own XML serialisation; output: OWL; plugin to NORMA (that is a plugin to Visual Studio); notes: demo paper at `https://ceur-ws.org/Vol-1660/demo-paper3.pdf` and reference paper at DOI 10.1007/978-3-319-48472-3_52.
- DOGMA Modeler; input: Object-Role Modelling models; standalone tool; uses the notion of vocabulary/lightweight ontology with conceptual data model/application profile with the constraints.

Bibliography

[1] model2owl project documentation, https://docs.ted.europa.eu/M2O/latest/index.html, last accessed: 12 January 2026

[2] model2owl project documentation, https://docs.ted.europa.eu/M2O/latest/transformation/uml2owl-transformation.html, last accessed: 12 January 2026

[3] Semic style guide, https://semiceu.github.io/style-guide/1.0.0/index.html, last accessed: 12 January 2026

[4] Semic style guide – conceptual model conventions (UML), https://semiceu.github.io/style-guide/1.0.0/gc-conceptual-model-conventions.html, last accessed: 12 January 2026

[5] Distributed ontology, model, and specification language (February 2018), http://www.omg.org/spec/DOL/

[6] Oslo modelleringsregels (2018), https://github.com/Informatievlaanderen/OSLO-handleiding/blob/master/Modellering/OSLO-Modelleringsregels.pdf, last accessed: 1 April 2026

[7] Interoperable specifications profile - version 1.0.0. Tech. rep. (Feb 2026), http://w3id.org/inspec/specification, last accessed: 12 March 2026

[8] Ahmetaj, S., Löhnert, B., Ortiz, M., Šimkus, M.: Magic shapes for shacl validation. Proc. VLDB Endow. **15**(10), 2284?2296 (2022). https://doi.org/10.14778/3547305.3547329

[9] Atkinson, R., Car, N.J.: The profiles vocabulary. Tech. rep., W3C (December 2019), https://www.w3.org/TR/dx-prof/, last accessed: 12 March 2026

[10] Baldassarre, C., Schiltz, A., Stani, E.: Semantic registry model (10 November 2025), https://semiceu.github.io//SRM/releases/1.0.0/, last accessed: 1 April 2026

[11] Bernabé, C.H., Keet, C.M., Khan, Z.C., Mahlaza, Z.: A method to improve alignments between domain and foundational ontologies. In: 14th International Conference on For-

mal Ontology in Information Systems 2023 (FOIS'23). FAIA, vol. 377, pp. 125–139. IOS Press (2023)

[12] Boger, M., Jeckle, M., Mueller, S., Fransson, J.: Diagram interchange for uml. In: Jézéquel, J.M., Hussmann, H., Cook, S. (eds.) UML 2002 — The Unified Modeling Language. pp. 398–411. Springer (2002)

[13] Booshehri, M., Emele, L., Flügel, S., Förster, H., Frey, J., Frey, U., Glauer, M., Hastings, J., Hofmann, C., Hoyer-Klick, C., Hülk, L., Kleinau, A., Knosala, K., Kotzur, L., Kuckertz, P., Mossakowski, T., Muschner, C., Neuhaus, F., Pehl, M., Robinius, M., Sehn, V., Stappel, M.: Introducing the open energy ontology: Enhancing data interpretation and interfacing in energy systems analysis. Energy and AI **5**, 100074 (2021). https://doi.org/10.1016/j.egyai.2021.100074

[14] Braun, G., Fillottrani, P.R., Keet, C.M.: A framework for interoperability between models with hybrid tools. Journal of Intelligent Information Systems **60**, 437–462 (2023)

[15] Brickley, D., Guha, R.: Rdf schema 1.1. W3c recommendation, W3C (February 2014), https://www.w3.org/TR/rdf-schema/, last accessed: 12 March 2026

[16] Cimiano, P., McCrae, J.P., Buitelaar, P.: Lexicon model for ontologies: Community report. Final community group report, 10 may 2016, W3C (2016)

[17] Cyganiak, R., Wood, D., Lanthaler, M.: Rdf 1.1 concepts and abstract syntax (February 2014), https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/

[18] Dawson, W.L., Keet, C.M.: Ontology pattern substitution: Toward their use for domain ontologies. In: da Silva Oliveira, I.J., et al. (eds.) Proceedings of the Joint Ontology Workshops (JOWO) - Episode X: The Tukker Zomer of Ontology, and satellite events co-located with the 14th International Conference on Formal Ontology in Information Systems (FOIS'24). vol. 3882, p. 13p. CEUR-WS (2024), https://ceur-ws.org/Vol-3882/demo-2.pdf, 15-19 July, 2024, Enschede, The Netherlands

[19] Digg Sweden: Inspec – interoperable specifications profile (documentation). Tech. rep. (2026), https://w3id.org/inspec, last accessed: 12 March 2026

[20] Fillotrani, P.R., Keet, C.M.: Evidence-based lean conceptual data modelling languages. Journal of Computer Science and Technology **21**(2), e10 (Oct 2021). https://doi.org/10.24215/16666038.21.e10

[21] Fillottrani, P.R., Keet, C.M.: Dimensions affecting representation styles in ontologies. In: 1st Iberoamerican conference on Knowledge Graphs and Semantic Web (KGSWC'19). CCIS, vol. 1029, pp. 186–200. Springer (2019), 24-28 June 2019, Villa Clara, Cuba

[22] Fillottrani, P.R., Keet, C.M.: An analysis of commitments in ontology language design. In: Brodaric, B., Neuhaus, F. (eds.) 11th International Conference on Formal Ontology in Information Systems 2020 (FOIS'20). FAIA, vol. 330, pp. 46–60. IOS Press (2020). https://doi.org/10.3233/FAIA200659

[23] Fillottrani, P.R., Keet, C.M.: Logics for Conceptual Data Modelling: A Review. Transactions on Graph Data and Knowledge **2**(1), 4:1–4:30 (2024). https://doi.org/10.4230/TGDK.2.1.4

[24] García-Castro, R., Lefrancois, M., Poveda-Villalón, M., Daniele, L.: The ETSI SAREF Ontology for Smart Applications: A Long Path of Development and Evolution, chap. 7, pp. 183–215. John Wiley & Sons, Ltd (2023). https://doi.org/10.1002/9781119899457.ch7

[25] Gennari, J.H., Musen, M.A., Fergerson, R.W., Grosso, W.E., Crubézy, M., Eriksson, H., Noy, N.F., Tu, S.W.: The evolution of Protégé: an environment for knowledge-based systems development. International Journal of Human-Computer Studies **58**(1), 89–123 (2003). https://doi.org/10.1016/S1071-5819(02)00127-1

[26] Guizzardi, G., Fonseca, C.M., Benevides, A.B., Almeida, J.P.A., Porello, D., Sales, T.P.: Endurant types in ontology-driven conceptual modeling: Towards OntoUML 2.0. In: Trujillo, J.C., et al. (eds.) Proc. of ER 2018. LNCS, vol. 11157, pp. 136–150. Springer (2018)

[27] Harmse, H.: UML Class Diagram to OWL and SROIQ Reference, `https://henrietteharmse.com/wp-content/uploads/2017/11/uml-class-diagram-to-owl-and-sroiq-reference.pdf`, last accessed: 6 February 2026

[28] Horridge, M., Bechhofer, S.: The OWL API: A java API for OWL ontologies. Semantic Web **2**(1), 11–21 (2011)

[29] Huang, S., Gohel, V., Hsu, S.: Towards interoperability of uml tools for exchanging high-fidelity diagrams. In: Proceedings of the 25th Annual ACM International Conference on Design of Communication. pp. 134–141. SIGDOC '07, ACM, New York, NY, USA (2007). https://doi.org/10.1145/1297144.1297172

[30] Isaac, A., Summers, E.: SKOS Simple Knowledge Organization System Primer. W3c standard, World Wide Web Consortium (August 2009), `http://www.w3.org/TR/skos-primer`

[31] Ke, J., Zacouris, Z., Acosta, M.: Efficient validation of SHACL shapes with reasoning. Proc. VLDB Endow. **17**(11), 3589–3601 (2024). https://doi.org/10.14778/3681954.3682023

[32] Keet, C.M.: An introduction to ontology engineering (2nd ed.), Computing, vol. 20. College Publications, UK (2025), 339p

[33] Keet, C.M., Fernández-Reyes, F.C., Morales-González, A.: Representing mereotopological relations in OWL ontologies with ONTOPARTS. In: Simperl, E., et al. (eds.) Proceedings of the 9th Extended Semantic Web Conference (ESWC'12). LNCS, vol. 7295, pp. 240–254. Springer (2012), 29-31 May 2012, Heraklion, Crete, Greece

[34] Keet, C.M., Fillottrani, P.R.: An ontology-driven unifying metamodel of UML Class Diagrams, EER, and ORM2. Data & Knowledge Engineering **98**, 30–53 (2015). https://doi.org/0.1016/j.datak.2015.07.004

[35] Keet, C.M., Kutz, O.: Orchestrating a network of mereo(topo)logical theories. In: Proceedings of the Knowledge Capture Conference (K-CAP'17). pp. 11:1–11:8. ACM, New York, NY, USA (2017). https://doi.org/10.1145/3148011.3148013

[36] Klímek', J.: Data specification vocabulary (DSV). Specification (9 January 2026), https://mff-uk.github.io/data-specification-vocabulary/dsv/, last accessed on 23 February 2026

[37] Knublauch, H., Kontokostas, D.: Shapes constraint language (SHACL). W3c recommendation, W3C (July 2017), https://www.w3.org/TR/shacl/, last accessed: 12 March 2026

[38] Kremen, P., Necaský', M.: Improving discoverability of open government data with rich metadata descriptions using semantic government vocabulary. Journal of Web Semantics **55**, 1–20 (2019). https://doi.org/10.1016/j.websem.2018.12.009

[39] Lamy, J.B.: Owlready: Ontology-oriented programming in python with automatic classification and high level constructs for biomedical ontologies. Artificial Intelligence in Medicine **80**, 11–28 (2017)

[40] Matentzoglu, N., Malone, J., Mungall, C., Stevens, R.: MIRO: guidelines for minimum information for the reporting of an ontology. Journal of Biomedical Semantics **9**, 6 (2018)

[41] McLaughlin, J., Lagrimas, J., Iqbal, H., Parkinson, H., Harmse, H.: Ols4: a new ontology lookup service for a growing interdisciplinary knowledge ecosystem. Bioinformatics **41**(5), btaf279 (05 2025). https://doi.org/10.1093/bioinformatics/btaf279

[42] Motik, B., Patel-Schneider, P.F., Parsia, B.: OWL 2 web ontology language structural specification and functional-style syntax. W3c recommendation, W3C (27 Oct 2009), http://www.w3.org/TR/owl2-syntax/

[43] Moxon, S.A.T., Solbrig, H., Harris, N.L., Kalita, P., Miller, M.A., Patil, S., Schaper, K., Bizon, C., Caufield, J.H., Cuesta, S.C., Cox, C., Dekervel, F., Dooley, D.M., Duncan, W.D., Fliss, T., Gehrke, S., Graefe, A.S.L., Hegde, H., Ireland, A.J., Jacobsen, J.O.B., Krishnamurthy, M., Kroll, C., Linke, D., Ly, R., Matentzoglu, N., Overton, J.A., Saunders, J.L., Unni, D.R., Vaidya, G., Vierdag, W.M.A.M., Contributors, L.C., Ruebel, O., Chute, C.G., Brush, M.H., Haendel, M.A., Mungall, C.J.: Linkml: an open data modeling framework. GigaScience **15**, giaf152 (2025). https://doi.org/10.1093/gigascience/giaf152

[44] van Nuffelen, B.: Dcat-ap 3.0.1. Application profile, W3C (10 July 2025), https://semiceu.github.io/DCAT-AP/releases/3.0.1/

[45] Object Management Group: Omg unified modeling language (omg uml). Standard 2.5.1, Object Management Group (2017), http://www.omg.org/spec/UML/2.5.1/

[46] OMG: XML Metadata Interchange (XMI) Specification (June 2015), https://www.omg.org/spec/XMI/2.5.1/About-XMI/, version 2.5.1

[47] Pazienza, M.T., Scarpato, N., Stellato, A., Turbati, A.: Semantic turkey: A browser-integrated environment for knowledge acquisition and management. Semantic Web Journal **3**(3), 279–292 (2012). https://doi.org/10.3233/SW-2011-0033

[48] Smith, B., Ashburner, M., Rosse, C., Bard, J., Bug, W., Ceusters, W., Goldberg, L., Eilbeck, K., Ireland, A., Mungall, C., OBI Consortium, T., Leontis, N., Rocca-Serra, A., Ruttenberg, A., Sansone, S.A., Shah, M., Whetzel, P., Lewis, S.: The OBO Foundry: Coordinated evolution of ontologies to support biomedical data integration. Nature Biotechnology **25**(11), 1251–1255 (2007)

[49] Stellato, A., Fiorelli, M., Turbati, A., Lorenzetti, T., van Gemert, W., Dechandon, D., Laaboudi-Spoiden, C., Gerencser, A., Waniart, A., Costetchi, E., Keizer, J.: Vocbench 3: A collaborative semantic web editor for ontologies, thesauri and lexicons. Semantic Web Journal **11**(5), 855–881 (2020). https://doi.org/10.3233/SW-200370

[50] Stenchlak, S., Klimek, J., Skoda, P., Necasky, M.: Dataspecer: Development of consistent semantic data specification ecosystems. Semantic Web Journal (under review) (2026), https://www.semantic-web-journal.org/content/dataspecer-development-consistent-semantic-data-specification-ecosystems-0, last accessed: 3 April 2026

[51] Suarez-Figueroa, M.C., de Cea, G.A., Buil, C., Dellschaft, K., Fernandez-Lopez, M., Garcia, A., Gómez-Pérez, A., Herrero, G., Montiel-Ponsoda, E., Sabou, M., Villazon-Terrazas, B., Yufei, Z.: NeOn methodology for building contextualized ontology networks. NeOn Deliverable D5.4.1, NeOn Project (2008)

[52] Whetzel, P.L., Noy, N.F., Shah, N.H., Alexander, P.R., Nyulas, C., Tudorache, T., Musen, M.A.: BioPortal: enhanced functionality via new web services from the national center for biomedical ontology to access and use ontologies in software applications. Nucleic Acids Research **39**(Web-Server-Issue) (2011)